

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FALTY OF COMPUTER SCIENCE AND ENGINEERING



CAPSTONE PROJECT COMPUTER ENGINEERING

Motion Planning around Obstacles

Semester 251

Instructor: Assoc. Prof. Le Hong Trang

STT	Full name	ID	Note
1	Nguyen Tran Dang Khoa	2211635	
2	Ho Dang Khoa	2211588	

Ho Chi Minh City, December 2025

Tóm tắt

Hoạch định chuyển động cho robot trong môi trường chứa vật cản vốn là một bài toán tối ưu hóa phi tuyến đầy thách thức do tính chất không lồi của không gian cấu hình. Trong khi các phương pháp dựa trên lấy mẫu truyền thống thường gặp hạn chế về tính tối ưu và khả năng xử lý các ràng buộc động lực học phức tạp, bài báo này đề xuất một phương pháp tiếp cận mới dựa trên Đồ thị các Tập Lồi (Graphs of Convex Sets - GCS). Bằng cách phân rã không gian tự do thành các vùng lồi hữu hạn và mô hình hóa quỹ đạo robot sử dụng đường cong Bézier, chúng tôi thiết lập bài toán dưới dạng tối ưu hóa hỗn hợp nguyên. Cuối cùng ta áp dụng kỹ thuật nới lỏng lồi (convex relaxation) chặt chẽ để chuyển đổi bài toán phức tạp này thành bài toán Lập trình nón bậc hai (Second-Order Cone Programming - SOCP) dễ giải. Kết quả thực nghiệm trên các hệ thống có số chiều cao chứng minh phương pháp này vượt trội hơn so với các thuật toán PRM truyền thống, giảm thiểu đáng kể thời gian tính toán trong khi vẫn đảm bảo tìm được quỹ đạo tối ưu toàn cục. Chúng tôi kiểm chứng GCS trên một loạt các môi trường với độ phức tạp tăng dần, từ các vật cản 2D đơn giản và mê cung phức tạp đến các thiết bị bay quadrotor động lực học và tay máy 7 bậc tự do (7-DoF) trong không gian nhiều chiều. Các thực nghiệm số chứng minh rằng GCS đạt hiệu năng vượt trội một cách nhất quán so với các bộ hoạch định dựa trên lấy mẫu được sử dụng rộng rãi (ví dụ: PRM). Cụ thể, trong các nhiệm vụ nhiều chiều, GCS giảm thời gian truy vấn trực tuyến lên đến một bậc so với PRM tiêu chuẩn, đồng thời tìm ra các quỹ đạo tối ưu toàn cục ngắn hơn đáng kể (giảm tới 40% độ dài) so với các bộ hoạch định dựa trên lấy mẫu đã được xử lý hậu kỳ [1].

Contents

1	Introduction	1
1.1	Motivation and Technical Challenges	1
1.2	Problem statement	3
1.3	Goals	4
1.4	Scope	4
1.5	Thesis structure	4
2	Theoretical Background	6
2.1	Convex Analysis and Optimization	6
2.1.1	Convex Sets	6
2.1.2	Convex Cones and Homogenization	7
2.1.2.1	Perspective Cones and Homogenization	7
2.1.2.2	Dual Cones	9
2.1.3	Convex Functions	9
2.1.4	Optimization Problems	10
2.1.4.1	Convex Optimization	10
2.1.4.2	Conic Optimization	10
2.1.4.3	Mixed-Integer Programs	11
2.2	Graph Theory and the Shortest Path Problem	12
2.2.1	Graph Theory Fundamentals	12
2.2.1.1	Basic Definitions and Concepts	12
2.2.1.2	Combinatorial Optimization on Graphs	12
2.2.2	The Shortest Path Problem (SPP)	13

2.2.2.1	Definition and Classical Algorithms . . .	13
2.2.2.2	Network Flow Formulation for the Short- est Path Problem	14
2.3	Kinodynamic Trajectory Generation	15
2.3.1	Kinodynamic Formulation	16
2.3.2	Smoothness and Continuity Constraints	16
2.3.3	Background on Bézier Curves	17
2.4	Convex Decomposition	19
2.4.1	Định nghĩa	19
2.4.2	Phân loại Phương pháp Phân hoạch Lỗi	20
2.4.2.1	Phân hoạch Lỗi Chính xác (Exact Con- vex Decomposition – ECD)	20
2.4.2.2	Phân hoạch Lỗi Xấp xỉ (Approximate Con- vex Decomposition – ACD)	21
3	Related works	22
3.1	Sampling-Based Methods	22
3.2	Discussion	22
4	Proposed Solution	23
4.1	Phân hoạch không gian an toàn	23
4.2	Các Phương pháp Cốt lõi cho Phân hoạch Không gian Tự do	24
4.2.1	Phân hoạch Lỗi Xấp xỉ (ACD) của Đa giác	24
4.2.1.1	Các Khái niệm Cơ bản trong Phân rã Lỗi	24
4.2.1.2	Ý tưởng và Chi tiết Thuật toán	25
4.2.1.3	Các Thước đo Độ lõm	27
4.2.1.4	Đặc tính của Thuật toán	28
4.2.2	Iterative Regional Inflation by Semi-Definite Pro- gramming (IRIS)	28

4.2.2.1	Ý tưởng và Chi tiết Thuật toán	28
4.2.2.2	Đặc tính của Thuật toán	34
4.2.3	Gieo mầm Tự động qua Lớp phủ Clique Khả kiến (VCC)	35
4.2.3.1	Ý tưởng và Chi tiết Thuật toán	36
4.2.3.2	Đặc tính và Tính tối ưu	39
4.3	Graphs of Convex Sets	39
4.3.1	Shortest Path Formulation in Graph of Convex Sets	39
4.3.2	Complexity Analysis	41
4.3.3	Detailed Formulation of Vertices and Edges . . .	42
4.3.4	Cost Function Specifications	43
4.3.4.1	Geometric Path Length	44
4.3.4.2	Dynamic Trajectory Costs	44
4.3.5	Mixed-Integer Convex Programming Formulation via Perspective Relaxation	45
4.3.6	Convexification of the GCS Problem via the RLT Technique	48
4.3.6.1	Theoretical Foundation: The Duality be- tween Perspective Cones and Valid In- equalities	48
4.3.6.2	Construction of Convex Constraints . .	49
4.3.6.3	Complete MICP Formulation	50
4.4	Proposed solution	52
4.5	Experiment setup	52
4.5.1	Implementation details	52
4.6	Results and analysis	52
5	Conclusion	53
5.1	Summary	53

5.2	Future Work	53
	References	54
	Appendix A ...	56
A.1		56
A.2		56

List of Figures

Figure 1.1	Boston Dynamics' Atlas and Amazon's Robin	2
Figure 2.1	Convex Sets Examples	6
Figure 2.2	Convex Hull Examples	7
Figure 4.1	(a) Nếu $x \in \partial P_i > 0$, hàm Resolve gộp biên ∂P_i vào đa giác P_0 . (b) Nếu $x \in \partial P_0$, hàm Resolve tách đa giác P thành hai đa giác P_1 và P_2 . (c) Độ lõm tại điểm x thay đổi sau khi đa giác được phân rã.	26
Figure 4.2	Quá trình đệ quy tiếp tục cho đến khi đạt đến giới hạn độ lõm đầu vào của người dùng τ	26
Figure 4.3	Giả sử r là điểm lõm có độ lõm lớn nhất. Sau khi giải quyết (resolve) r , độ lõm của s tăng lên. Nếu $\text{concave}(r) < \tau$, vùng s sẽ không bao giờ được xử lý, ngay cả khi $\text{concave}(s)$ lớn hơn τ	27
Figure 4.4	Độ lõm SL có thể xử lý được túi trong (a) một cách chính xác vì không có hướng chuẩn nào của các đỉnh trong túi ngược với hướng chuẩn của cầu. Tuy nhiên, túi trong (b) có thể dẫn đến độ lõm giảm không đơn điệu.	28
Figure 4.5	Siêu phẳng Phân cách	29
Figure 4.6	Ellipsoid Nội tiếp Cực đại	32

Figure 4.7	Lắp lại	33
Figure 4.8	Độ phức tạp thời gian	34
Figure 4.9	Lấy mẫu & Xây dựng Đồ thị Khả kiến (Visibility Graphs)	36
Figure 4.10	Tìm Lớp phủ Clique	37
Figure 4.11	Khởi tạo Ellipsoid	37
Figure 4.12	Lấp đầy thành Đa diện	38
Figure 4.13	Lắp lại	38

List of Tables

Chapter 1

Introduction

In chapter 1, the overview, objectives and goals of the research's project are illustrated. The outline of the report is also presented.

1.1 Motivation and Technical Challenges

Making optimal decisions in real-time is a cornerstone of modern engineering. The necessity of tightly coupling **discrete decisions** with **continuous control** is vividly illustrated in advanced robotics:

- **Humanoid Robots (e.g., Boston Dynamics' Atlas):** Navigating complex terrain, such as a parkour course, requires the robot to simultaneously select footstep locations (discrete) and compute the trajectories for its center of mass and limbs (continuous). Currently, solutions often rely on “hand-coding” the discrete components, which severely limits the robot's autonomy in novel environments [2].
- **Industrial Manipulators (e.g., Amazon's Robin):** Sorting efficiency depends on jointly optimizing the sequence of bins (discrete) and the arm's trajectory (continuous) [3]. A mere **1% increase** in operating speed through superior optimization translates to millions of additional packages processed annually.

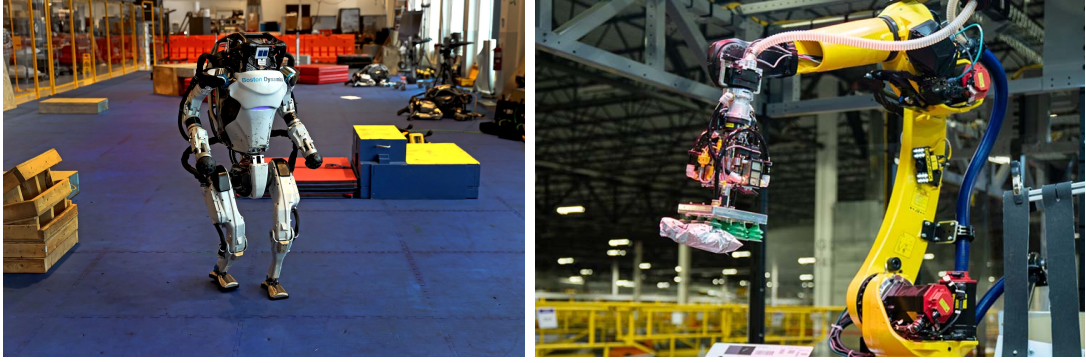


Figure 1.1: ***Left:** Boston Dynamics’ Atlas robot performing parkour, requiring discrete footstep planning and continuous trajectory optimization. **Right:** Amazon’s Robin robotic arm sorting packages, where discrete bin selection and continuous arm trajectory must be optimized jointly.*

However, solving these coupled problems simultaneously remains an open challenge, typically necessitating manual engineering interventions or settling for suboptimal solutions.

Algorithmically, selecting a motion planning method currently forces researchers to compromise between conflicting features: dimensionality, dynamic constraints, and completeness.

- **Trajectory Optimization:** These methods excel at handling robot dynamics and high-dimensional spaces. However, when facing obstacle-cluttered environments—which render the problem inherently non-convex—they often get trapped in **local minima**, failing to identify a collision-free trajectory.[4]
- **Sampling-based Planners:** To overcome these limitations, the robotics community frequently resorts to sampling-based methods (e.g., RRT, PRM) due to their “probabilistic completeness”. However, this comes at a cost: the resulting trajectories are often significantly **suboptimal**, and imposing **continuous differential constraints** on discrete samples remains notoriously difficult.[5], [6]

Mixed-Integer Convex Programming (MICP) has emerged as a promis-

ing solution that bridges this gap. It combines the strengths of both worlds: the completeness of sampling-based algorithms and the dynamic handling capabilities of trajectory optimization, while guaranteeing **global optimality** within a unified framework. Nevertheless, the adoption of MICP is severely limited by its prohibitive computational costs, often requiring minutes to solve even small-scale problems.

The objective of this research is to break this computational barrier. We focus on a crucial class of motion planning problems involving differential constraints and propose an MICP-based planner capable of reliably solving high-dimensional problems in a matter of seconds through a **Single Convex Program**.

1.2 Problem statement

In the ever-evolving world of technology, the concept of an e-commerce website employing a recommender system to customize the user experience is not unfamiliar, particularly in the gadget and computer sector. Despite the prevalence of e-commerce platforms as standard marketing and sales channels for these firms, there is a need to improve the current methods of user interaction, specifically in facilitating users to effectively retrieve their desired products. While these websites offer a wide range of products, the existing user interface and navigation system present challenges for customers to quickly and efficiently find specific computers and gadgets, resulting in a subpar user experience.

The current user interaction design of e-commerce websites in the computer and gadgets sector lacks the necessary features and functionalities to enhance the browsing and search experience. Users often struggle to locate their desired products due to limited search filters, inadequate categorization, and a lack of intuitive navigation options. This leads to frustration,

increased browsing time, and a potential loss of interest in exploring the full range of offerings.

One of the specific challenges lies in the use of filters to narrow down search results, which often requires users to possess a certain level of domain knowledge in gadgets, electronics, or computers. This can cause confusion for users who are new to this field and may not be familiar with the specific terminology or technical specifications. Similarly, searching using full-text search poses difficulties, as users need to have a good understanding of how to formulate effective search queries to retrieve products that meet their needs.

In summary, by empowering users to find products that meet their needs without requiring extensive technical expertise or struggling to formulate what they need in proper words, the websites can attract a broader customer base, drive conversion rates, and establish a competitive advantage in the market.

1.3 Goals

1.4 Scope

1.5 Thesis structure

There are five chapters in this capstone project:

- Chapter 1 briefly describes the project about problem's motivation, as well as the project's objectives and scope
- Chapter 2 is dedicated to presenting the foundational knowledge about convex optimization, mixed-integer optimization, and graph theory

that are essential for understanding the proposed method.

- Chapter 3 arranges the discussion of related works on the same task to deepen the understanding of existing methods and their constraints.
- Chapter 4 describes the GCS framework and the formulation of our MICP. From that, we present the experimental results and analysis of our method in various environments with different complexity levels.
- Chapter 5 summarizes whole project and the plan for future development.

Chapter 2

Theoretical Background

In Chapter 2, it presents about preliminary knowledge in this project.

2.1 Convex Analysis and Optimization

2.1.1 Convex Sets

Before discussing optimization problems, we establish the fundamental geometric objects used in this work. A set $\mathcal{C} \subseteq \mathbb{R}^n$ is defined as *convex* if, for any two points within the set, the line segment connecting them lies entirely within the set.

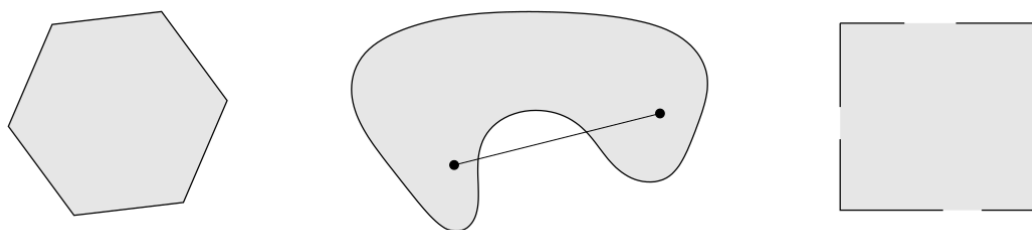


Figure 2.1: Examples of convex and nonconvex sets. Left: The hexagon (including boundary) is convex. Middle: The kidney-shaped set is nonconvex, as the connecting line segment exits the set. Right: The square with partial boundary inclusion is not convex.[7]

Closely related to convexity is the concept of the **convex hull**. The convex

hull of a set of points $\{x_1, x_2, \dots, x_m\} \subseteq \mathbb{R}^n$ is the smallest convex set containing those points, formally defined as the set of all convex combinations:

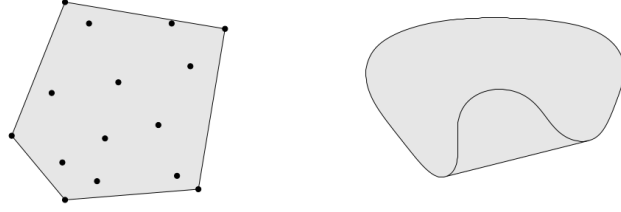


Figure 2.2: The convex hull of a set of points (depicted as the shaded area).[7]

In the context of motion planning and this thesis, we primarily focus on two specific types of convex sets:

- **Polyhedra:** Defined as the intersection of a finite number of half-spaces, represented as $\mathcal{P} = \{x \in \mathbb{R}^n \mid Ax \leq b\}$. When a polyhedron is bounded, it is referred to as a *polytope*.
- **Ellipsoids:** Defined as the image of a unit ball under an affine transformation. These are typically represented as $\mathcal{E} = \{x \in \mathbb{R}^n \mid (x - c)^\top Q^{-1}(x - c) \leq 1\}$, where Q is a positive definite matrix ($Q \succ 0$).

2.1.2 Convex Cones and Homogenization

A subset $\mathcal{K} \subseteq \mathbb{R}^n$ is a *cone* if it is closed under positive scalar multiplication (i.e., $\theta x \in \mathcal{K} \forall x \in \mathcal{K}, \theta \geq 0$). A set is a *convex cone* if it is both convex and a cone. Two critical concepts regarding convex cones are essential for this work: perspective cones (constructed via homogenization) and dual cones.

2.1.2.1 Perspective Cones and Homogenization

Homogenization is a powerful technique in convex analysis used to transform a general convex set into a cone in a higher-dimensional space. This process effectively “lifts” a set $\mathcal{S} \subseteq \mathbb{R}^n$ into $\mathbb{R}^{n+1}$ by introducing an auxiliary

dimension y . The homogenization $\tilde{\mathcal{S}}$ is defined as:

$$\tilde{\mathcal{S}} := \{(x, y) \in \mathbb{R}^{n+1} \mid y > 0, x \in y\mathcal{S}\}. \quad (2.1)$$

Crucially, $\tilde{\mathcal{S}}$ is convex if and only if the original set $\mathcal{S}$ is convex. This transformation allows for the representation of convex combinations within a conic structure, which is particularly useful for converting affine constraints into conic forms in optimization.

When a convex set $\mathcal{C}$ is described in a standard conic form, its homogenization is computationally straightforward. Observing that for $y > 0$, the set $y\mathcal{C}$ satisfies $\{yx \mid Ax + b \in \mathcal{K}\} = \{x' \mid Ax' + by \in \mathcal{K}\}$, we derive the *perspective cone*:

$$\tilde{\mathcal{C}} = \{(x, y) \in \mathbb{R}^{n+1} \mid y > 0, Ax + by \in \mathcal{K}\}. \quad (2.2)$$

Of significant practical relevance is the closure of this set, $\text{cl}(\tilde{\mathcal{C}}) = \{(x, y) \mid y \geq 0, Ax + by \in \mathcal{K}\}$. This closure extends the definition to include the boundary where $y=0$, which is critical for modeling logical constraints in mixed-integer programming.

For a compact set $\mathcal{C}$, as the scalar y approaches zero, the scaled set $y\mathcal{C}$ continuously shrinks towards the origin. Consequently, in the limit where $y = 0$, the condition $(x, 0) \in \text{cl}(\tilde{\mathcal{C}})$ implies that x must be the zero vector (assuming $\mathcal{C}$ contains the origin and is bounded). This geometric behavior effectively functions as a logical switch:

- When $y > 0$, the variable x is constrained to a scaled version of the set $\mathcal{C}$.
- When $y = 0$, the variable x is forced to the origin ($x = 0$).

This property is directly exploited in our application to Shortest Path Problems in Graphs of Convex Sets (GCS). In the Mixed-Integer Convex Programming (MICP) formulation, the binary activation variable for an edge

serves as the scaling factor y . When an edge is inactive ($y = 0$), the associated state contributions are forced to zero, and the cost—modeled via the perspective function—naturally vanishes. The perspective function $\tilde{\ell}_e$ is defined to return 0 when both $y = 0$ and the state variables are 0, ensuring that inactive components incur no cost in the optimization objective.

2.1.2.2 Dual Cones

The second essential concept is the *dual cone*. For a convex cone $\mathcal{K} \subseteq \mathbb{R}^n$, the dual cone $\mathcal{K}^*$ consists of all vectors making a non-negative inner product with every vector in $\mathcal{K}$:

$$\mathcal{K}^* := \{y \in \mathbb{R}^n \mid x^\top y \geq 0, \forall x \in \mathcal{K}\}. \quad (2.3)$$

A fundamental property is that $\mathcal{K}^*$ is always convex, regardless of whether $\mathcal{K}$ is convex. We frequently encounter three specific classes of cones, all of which are *self-dual* (i.e., $\mathcal{K}^* = \mathcal{K}$):

1. The **non-negative orthant**: $\mathbb{R}_+^n = \{x \in \mathbb{R}^n \mid x_i \geq 0\}$.
2. The **second-order cone** (Lorentz cone): $\mathcal{Q}^n = \{(t, x) \in \mathbb{R} \times \mathbb{R}^{n-1} \mid \|x\|_2 \leq t\}$.
3. The **positive semidefinite cone**: $\mathcal{S}_+^n = \{X \in \mathcal{S}^n \mid X \succeq 0\}$.

2.1.3 Convex Functions

A function $f: \mathcal{D} \rightarrow \mathbb{R}$ is *convex* if its domain $\mathcal{D} \subseteq \mathbb{R}^n$ is a convex set and if, for all $x, y \in \mathcal{D}$ and $\theta \in [0, 1]$, the following inequality holds:

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y). \quad (2.4)$$

Geometrically, the line segment connecting any two points on the function's graph lies above or on the graph. A function is *strictly convex* if this

inequality holds strictly for $x \neq y$ and $\theta \in (0, 1)$. Conversely, f is *concave* if $-f$ is convex.

The most critical analytical property of convex functions in optimization is that **any local minimum is guaranteed to be a global minimum**.

2.1.4 Optimization Problems

2.1.4.1 Convex Optimization

A **Convex Program (CP)** is formulated as:

$$\text{minimize} \quad f(x) \tag{2.5a}$$

$$\text{subject to} \quad x \in \mathcal{C}, \tag{2.5b}$$

where the objective function f and the feasible set $\mathcal{C}$ are both convex. A point x^{opt} is an *optimal solution* if it is feasible and $f(x^{\text{opt}}) \leq f(x)$ for all $x \in \mathcal{C}$.

While theoretical convexity does not strictly guarantee efficiency (e.g., checking non-negativity of polynomials is NP-hard despite the set being a convex cone), for the purpose of this thesis, we use “convex optimization problem” as a synonym for “efficiently solvable optimization problem.” The majority of practical CPs can be solved reliably using interior-point methods.

2.1.4.2 Conic Optimization

A **Conic Program (KP)** is a subclass of CP with a linear objective and constraints defined by a convex cone $\mathcal{K}$:

$$\text{minimize} \quad c^\top x \tag{2.6a}$$

$$\text{subject to} \quad Ax + b \in \mathcal{K}. \tag{2.6b}$$

Specific classes of KPs include:

- **Linear Programs (LP)**: $\mathcal{K}$ is the non-negative orthant.
- **Second-Order Cone Programs (SOCP)**: $\mathcal{K}$ is the product of second-order cones.
- **Semidefinite Programs (SDP)**: $\mathcal{K}$ is the semidefinite cone.

These classes represent a hierarchy of modeling power: $LP \subset SOCP \subset SDP$. Another common class is the **Quadratic Program (QP)**, which has a quadratic objective and polyhedral constraints. While every QP can be formulated as an SOCP, specialized active-set algorithms for QPs are often more effective.

2.1.4.3 Mixed-Integer Programs

Mixed-Integer Programs (MIPs) generalize optimization by partitioning variables into continuous vector $x \in \mathbb{R}^n$ and integer vector $z \in \mathbb{Z}^m$:

$$\text{minimize} \quad f(x, z) \tag{2.7a}$$

$$\text{subject to} \quad (x, z) \in \mathcal{C}, \tag{2.7b}$$

$$z \in \mathbb{Z}^m. \tag{2.7c}$$

The integrality constraint renders the feasible set non-convex and discrete, typically making MIPs NP-hard. Standard methods relying on differentiability are inapplicable.

To solve MIPs, **relaxation** is commonly employed (e.g., replacing $z_j \in \{0, 1\}$ with $0 \leq z_j \leq 1$). The optimal value of the relaxed problem provides a lower bound on the MIP solution. If the relaxed solution is not integer-feasible, algorithms like **Branch-and-Bound** are used. This method systematically partitions the search space into sub-regions (branches) to eliminate fractional values, exploring a search tree to find the global optimum.

2.2 Graph Theory and the Shortest Path Problem

2.2.1 Graph Theory Fundamentals

2.2.1.1 Basic Definitions and Concepts

Formally, a graph is defined as an ordered pair $G = (\mathcal{V}, \mathcal{E})$, comprising a set of vertices $\mathcal{V}$ and a set of edges $\mathcal{E}$, where each edge connects a pair of vertices. The structural classification of the graph depends on the ordering of these pairs: unordered pairs $\{u, w\}$ denote an undirected graph, whereas ordered pairs (u, w) characterize a directed graph. In the context of optimization, the graph is typically augmented with a cost function $c : \mathcal{E} \rightarrow \mathbb{R}$, assigning a real-valued weight to each edge.

Key topological properties include the vertex degree, which counts incident edges; in directed graphs, this is distinguished into in-degree ($\deg^-$) and out-degree ($\deg^+$). Connectivity is described via paths—finite sequences of distinct vertices and edges—where the path length is the aggregate cost of its constituent edges. A cycle is defined as a path that originates and terminates at the same vertex. Optimization problems generally operate on a search domain defined by subgraphs $H = (W, F)$, formed by subsets of vertices $W \subseteq \mathcal{V}$ and edges $F \subseteq \mathcal{E}$ that maintain incidence relationships.

2.2.1.2 Combinatorial Optimization on Graphs

Combinatorial optimization problems on graphs generally seek a substructure that minimizes an objective function subject to validity constraints. For a weighted graph $G = (\mathcal{V}, \mathcal{E})$ and a set of feasible subgraphs $\mathcal{H}$, the problem is to identify a subgraph $H = (W, F) \in \mathcal{H}$ that minimizes the cumulative weight of its components. This is formally expressed as:

$$\text{minimize} \quad \sum_{v \in W} c_v + \sum_{e \in F} c_e \quad (1.1a)$$

$$\text{subject to} \quad H = (W, F) \in \mathcal{H}. \quad (1.1b)$$

To utilize standard mathematical programming techniques, such as Branch and Bound, the problem is transformed into an Integer Linear Programming (ILP) formulation. This involves parameterizing the search space using incidence vectors $\mathbf{y}^H \in \{0, 1\}^{|\mathcal{V} \cup \mathcal{E}|}$, where binary variables indicate the inclusion of specific vertices or edges in the solution. Geometrically, the set of valid subgraphs corresponds to a polytope $\mathcal{Y}$ in Euclidean space. The intersection of this polytope with the integer grid, $\mathcal{Y} \cap \{0, 1\}^{|\mathcal{V} \cup \mathcal{E}|}$, yields the incidence vectors of feasible subgraphs. Consequently, the discrete optimization problem is recast as a linear optimization over an integer domain:

$$\text{minimize} \quad \sum_{v \in \mathcal{V}} c_v y_v + \sum_{e \in \mathcal{E}} c_e y_e \quad (1.2a)$$

$$\text{subject to} \quad \mathbf{y} \in \mathcal{Y} \cap \{0, 1\}^{|\mathcal{V} \cup \mathcal{E}|}. \quad (1.2b)$$

This formulation allows for the application of convex analysis, ensuring that the optimal solution to the algebraic model corresponds exactly to the optimal substructure in the original graph problem.

2.2.2 The Shortest Path Problem (SPP)

2.2.2.1 Definition and Classical Algorithms

The Shortest Path Problem (SPP) on a weighted graph $G = (\mathcal{V}, \mathcal{E})$ involves identifying a path P between a source s and a destination t such that the summation of edge weights is minimized. If P comprises a sequence of edges

$e_1, \dots, e_k$, the objective is to minimize $\sum_{i=1}^k c_{e_i}$.

Classical approaches to solving the SPP vary based on edge weight characteristics. Dijkstra's algorithm employs a greedy strategy suitable for graphs with non-negative weights ($c_e \geq 0$). For graphs containing negative edge weights, provided no negative cycles exist, the Bellman-Ford algorithm is utilized, albeit with higher computational complexity. The Floyd-Warshall algorithm extends this to the all-pairs shortest path problem, accommodating negative edges but remaining sensitive to negative cycles.

2.2.2.2 Network Flow Formulation for the Shortest Path Problem

The Shortest Path Problem on a directed graph $G = (\mathcal{V}, \mathcal{E})$ with non-negative edge costs $c_{uv} \geq 0$ can be rigorously modeled as a Minimum Cost Flow problem involving a single unit of flow. We define binary decision variables $x_{uv} \in \{0, 1\}$ for each edge $(u, v) \in \mathcal{E}$, where $x_{uv} = 1$ indicates the edge is part of the optimal path. The optimization model is formulated as follows:

$$\text{minimize} \quad \sum_{(u,v) \in \mathcal{E}} c_{uv} x_{uv} \quad (2.8a)$$

$$\text{subject to} \quad \sum_{v \in \mathcal{V}^+(u)} x_{uv} - \sum_{v \in \mathcal{V}^-(u)} x_{vu} = \begin{cases} 1 & \text{if } u = s \\ -1 & \text{if } u = t \\ 0 & \text{otherwise} \end{cases}, \quad \forall u \in \mathcal{V} \quad (2.8b)$$

$$\sum_{v \in \mathcal{V}^-(u)} x_{vu} \leq 1, \quad \forall u \in \mathcal{V} \setminus \{s, t\} \quad (2.8c)$$

$$x_{uv} \geq 0, \quad \forall (u, v) \in \mathcal{E} \quad (2.8d)$$

Here, $\mathcal{V}^+(u)$ and $\mathcal{V}^-(u)$ denote the sets of successors and predecessors of

node u , respectively. The constraints in this formulation enforce specific structural properties. Equation (4.8b) represents flow conservation (Kirchhoff’s law), ensuring that a net flow of one unit leaves the source s , enters the target t , and is conserved at all intermediate nodes. Equation (2.8c) imposes a node capacity constraint, restricting the inflow at any intermediate vertex to unity, thereby enforcing a simple path structure where no vertex is revisited. Given non-negative costs, cycles are inherently suboptimal, rendering explicit cycle-elimination constraints unnecessary.

A significant advantage of this formulation lies in the algebraic properties of the constraint matrix derived from (4.8b). This node-arc incidence matrix exhibits Total Unimodularity (TUM). According to combinatorial optimization theory, if the constraint matrix is TUM and the right-hand side vector is integral, every basic feasible solution of the Linear Programming (LP) relaxation is guaranteed to be integral [8]. Consequently, despite the continuous non-negativity constraints in (2.8d), the problem can be solved efficiently using standard LP algorithms to yield a valid binary path, obviating the need for computationally intensive integer programming methods.

2.3 Kinodynamic Trajectory Generation

This section outlines the mathematical formulation for generating kinodynamically feasible trajectories. Unlike geometric path planning, which solely considers collision-free configurations in the workspace, motion planning (kinodynamic) accounts for the system’s differential constraints, such as velocity, acceleration, and jerk limits. We leverage Bézier curves as the trajectory parameterization scheme, which is particularly advantageous for the Graph of Convex Sets (GCS) framework due to its convexity properties.

2.3.1 Kinodynamic Formulation

Consider a robotic system whose state evolution is governed by a set of ordinary differential equations (ODEs):

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t)), \quad \mathbf{x}(t) \in \mathcal{X}, \mathbf{u}(t) \in \mathcal{U} \quad (2.9)$$

where $\mathbf{x}(t) \in \mathbb{R}^n$ represents the state vector (typically comprising position, velocity, and acceleration), and $\mathbf{u}(t) \in \mathbb{R}^m$ denotes the control inputs. The goal is to compute a trajectory $\sigma(t) : [0, T] \rightarrow \mathcal{X}$ that minimizes a specific cost functional while satisfying:

1. **Boundary Conditions:** The start and goal states $\mathbf{x}(0) = \mathbf{x}_{start}$ and $\mathbf{x}(T) = \mathbf{x}_{goal}$.
2. **Safety Constraints:** $\sigma(t) \in \mathcal{X}_{free}$ for all t , ensuring collision avoidance.
3. **Dynamic Limits:** Inequality constraints on higher-order derivatives to ensure physical feasibility:

$$|\dot{\sigma}(t)| \leq v_{max}, \quad |\ddot{\sigma}(t)| \leq a_{max}, \quad |\dddot{\sigma}(t)| \leq j_{max} \quad (2.10)$$

2.3.2 Smoothness and Continuity Constraints

In the GCS framework, a full trajectory is composed of multiple piecewise Bézier segments. To ensure smooth motion across the boundaries of adjacent convex sets, continuity constraints must be enforced at the transition points. Let $r_A(t)$ defined by control points $\{a_0, \dots, a_n\}$ and $r_B(t)$ defined by $\{b_0, \dots, b_n\}$ be two consecutive segments.

- **C^0 Continuity (Position):** Ensures the path is connected.

$$a_n = b_0 \quad (2.11)$$

- **C^1 Continuity (Velocity):** Ensures continuous velocity, preventing infinite acceleration spikes.

$$a_n - a_{n-1} = b_1 - b_0 \quad (2.12)$$

- **C^2 Continuity (Acceleration):** Ensures continuous force/torque application, which is crucial for minimizing mechanical wear and jerk.

$$(a_n - 2a_{n-1} + a_{n-2}) = (b_2 - 2b_1 + b_0) \quad (2.13)$$

By enforcing these equality constraints at the edges of the graph, we generate a globally smooth trajectory that respects the underlying physics of the robot.

2.3.3 Background on Bézier Curves

In order to tackle the kinodynamic planning problem numerically, it is necessary to parameterize the trajectory functions through a finite number of decision variables. To this end, we employ Bézier curves. This section recalls the definition and the basic properties of this family of curves.

A Bézier curve is constructed using Bernstein polynomials. The k -th Bernstein polynomial of degree d , with $k = 0, \dots, d$, is defined as:

$$\beta_{k,d}(s) := \binom{d}{k} s^k (1-s)^{d-k}, \quad s \in [0, 1] \quad (2.14)$$

Note that the Bernstein polynomials of degree d are nonnegative and, by the binomial theorem, they sum up to one. Therefore, for each fixed $s \in [0, 1]$, the scalars $\{\beta_{k,d}(s)\}_{k=0}^d$ can be thought of as the coefficients of a convex combination. Bézier curves are obtained using these coefficients to combine

a given set of $d+1$ control points $\gamma_k \in \mathbb{R}^n$:

$$\gamma(s) := \sum_{k=0}^d \beta_{k,d}(s) \gamma_k \quad (2.15)$$

It is easily verified that Bézier curves enjoy the following properties, which are instrumental for the GCS framework:

- **Endpoint values:** The curve γ starts at the first control point and ends at the last control point:

$$\gamma(0) = \gamma_0 \quad \text{and} \quad \gamma(1) = \gamma_d \quad (2.16)$$

This property is essential for enforcing continuity constraints (C^0) when stitching multiple Bézier segments together in the graph.

- **Convex hull property:** The curve γ is entirely contained in the convex hull of its control points:

$$\gamma(s) \in \text{conv}(\{\gamma_k\}_{k=0}^d), \quad \forall s \in [0, 1] \quad (2.17)$$

In the context of GCS, if a convex region X_v represents a safe set, guaranteeing collision avoidance is reduced to constraining all control points to lie within that set:

$$\gamma_k \in X_v, \quad \forall k \in \{0, \dots, d\} \implies \gamma(s) \in X_v \quad (2.18)$$

This allows us to enforce continuous safety constraints through a finite set of linear inclusion constraints.

- **Derivative (Hodograph):** The derivative $\dot{\gamma}$ of the curve γ is also a Bézier curve, but of degree $d-1$, with control points defined by the

difference of adjacent points:

$$\dot{\gamma}(s) = \sum_{k=0}^{d-1} \beta_{k,d-1}(s) \dot{\gamma}_k \quad (2.19)$$

where $\dot{\gamma}_k = d(\gamma_{k+1} - \gamma_k)$ for $k = 0, \dots, d-1$. Similarly, higher-order derivatives (acceleration, jerk) can be expressed as linear combinations of the control points. Consequently, kinodynamic constraints (e.g., velocity limits $|\dot{\gamma}(s)| \leq v_{max}$) can be imposed as linear constraints on the differences between adjacent control points, preserving the convexity of the optimization problem.

- **Integral of convex function:** For a convex function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ (often representing the cost function in motion planning, such as energy or path length), the integral along the curve is bounded by the convex combination of the function values at the control points:

$$\int_0^1 f(\gamma(s)) ds \leq \frac{1}{d+1} \sum_{k=0}^d f(\gamma_k) \quad (2.20)$$

This inequality provides a convex upper bound on the integral cost, facilitating efficient optimization within the GCS mixed-integer convex programming formulation.

2.4 Convex Decomposition

This section provides an overview of convex decomposition, a fundamental technique used in computational geometry and physics simulations.

2.4.1 Định nghĩa

Phân hoạch lồi (Convex Decomposition) là quá trình chia một hình học phức tạp (ví dụ: đa giác hoặc đa diện không lồi) thành một tập hợp

các **vùng con lỗi** sao cho hợp của các vùng này khôi phục lại toàn bộ hình ban đầu và các vùng con không chồng lấn nhau, ngoại trừ biên chung.

Về mặt hình thức, với một miền hình học $P \subset \mathbb{R}^n$, một *phân hoạch lỗi* của P là tập hợp $\{P_1, P_2, \dots, P_k\}$ sao cho:

$$P = \bigcup_{i=1}^k P_i, \quad P_i \cap P_j = \partial P_i \cap \partial P_j \text{ với mọi } i \neq j,$$

và mỗi P_i là một tập lỗi. Mục tiêu là tìm tập phân hoạch này như là số lượng các vùng lỗi tối thiểu, giảm thiểu phần chồng chéo hay chất lượng của vùng lỗi được tạo ra (tránh quá "mảnh" hoặc "dài").

2.4.2 Phân loại Phương pháp Phân hoạch Lỗi

Trong lĩnh vực hình học tính toán và hoạch định chuyển động, các phương pháp phân hoạch lỗi thường được chia thành hai loại chính: **Phân hoạch Lỗi Chính xác (Exact Convex Decomposition – ECD)** và **Phân hoạch Lỗi Xấp xỉ (Approximate Convex Decomposition – ACD)**. Sự khác biệt giữa hai loại này phản ánh sự đánh đổi cơ bản giữa *tính chính xác hình học* và *hiệu quả tính toán*.

2.4.2.1 Phân hoạch Lỗi Chính xác (Exact Convex Decomposition – ECD)

Phân hoạch Lỗi Chính xác (ECD) là quá trình chia một hình dạng không lỗi thành các **thành phần con hoàn toàn lỗi**, không chồng lấn, sao cho hợp của chúng tái tạo lại chính xác hình dạng ban đầu. Mục tiêu thường là tối thiểu hóa số lượng vùng lỗi.

Tuy nhiên, ECD gặp hai thách thức lớn:

- **Độ phức tạp tính toán cao:** Bài toán tìm phân hoạch lỗi tối thiểu đã được chứng minh là NP-hard đối với đa giác có lỗ, nên không tồn tại thuật toán hiệu quả cho các trường hợp tổng quát.

- **Tính khả dụng hạn chế:** Ngay cả khi tính toán được, ECD thường tạo ra số lượng vùng rất lớn, khiến việc lưu trữ, xử lý và tối ưu hóa tiếp theo trở nên tốn kém.

Do đó, các phương pháp ECD chủ yếu mang tính lý thuyết, dùng làm chuẩn so sánh, trong khi hầu hết các ứng dụng thực tế trong robot và mô phỏng đều dựa vào các phương pháp xấp xỉ.

2.4.2.2 Phân hoạch Lỗi Xấp xỉ (Approximate Convex Decomposition – ACD)

Trước những giới hạn của ECD, các phương pháp **Phân hoạch Lỗi Xấp xỉ (ACD)** được phát triển để đạt sự cân bằng giữa độ chính xác và hiệu quả. ACD cho phép các thành phần không hoàn toàn lồi, mà chỉ “*gần lồi*” trong một mức dung sai lồi τ do người dùng xác định hoặc chỉ bao phủ một phần không gian tự do. Triết lý cốt lõi của ACD là chấp nhận một **mức độ lồi nhỏ có kiểm soát** để đổi lấy:

- **Hiệu quả tính toán cao hơn:** Các thuật toán ACD, chẳng hạn như phương pháp của Lien và Amato, chạy nhanh hơn nhiều so với ECD. [9]
- **Giảm mạnh số lượng vùng:** Cho phép biểu diễn hình dạng với ít vùng lồi hơn, đơn giản hơn và dễ xử lý hơn.
- **Kiểm soát mức độ chi tiết:** Một số thuật toán ACD như phương pháp của Lien và Amato cho phép người dùng điều chỉnh tham số lồi τ để cân bằng giữa độ chính xác và số lượng vùng lồi. Hay đối với phương pháp Visibility Clique Cover (VCC), nó cho phép người dùng chọn chỉ số để thể hiện mức độ bao phủ của vùng lồi so với không gian tự do ban đầu. [10]

Chapter 3

Related works

...

3.1 Sampling-Based Methods

3.2 Discussion

In this chapter, we have seen three distinct approaches to enhance...

Chapter 4

Proposed Solution

...

4.1 Phân hoạch không gian an toàn

Thay vì tiếp cận bài toán tránh vật cản theo cách truyền thống là mô tả các ràng buộc không lỗi dựa trên bề mặt vật cản, nhóm tác giả đề xuất một phương pháp chuyển đổi không gian cấu hình tự do (collision-free configuration space) thành hợp của một tập hợp hữu hạn các vùng lỗi bị chặn (bounded convex regions). Trong mô hình này, các vùng lỗi có thể chồng lấn lên nhau, và bài toán hoạch định chuyển động được phát biểu lại dưới dạng một chương trình quy hoạch lồi nguyên hỗn hợp (Mixed-Integer Convex Program - MICP). Cụ thể, các biến nguyên được sử dụng để gán từng đoạn quỹ đạo đa thức (polynomial trajectory segments) vào các vùng lỗi an toàn cụ thể này. Phương pháp này mang lại lợi thế lớn về mặt tính toán vì số lượng biến nguyên chỉ phụ thuộc vào số lượng vùng an toàn được tạo ra (sử dụng thuật toán IRIS) thay vì phụ thuộc vào số lượng mặt của vật cản, giúp giải quyết hiệu quả các môi trường phức tạp với hàng trăm vật cản. Hơn nữa, việc ràng buộc quỹ đạo nằm trọn trong các tập lỗi đảm bảo an toàn tuyệt đối cho toàn bộ hành trình liên tục, khắc phục nhược điểm "cắt góc" (corner cutting) thường gặp ở các phương pháp chỉ kiểm tra

va chạm tại các điểm lấy mẫu rời rạc.

4.2 Các Phương pháp Cốt lõi cho Phân hoạch Không gian Tự do

Để giải quyết thách thức về tính toán của phân hoạch tối ưu, một số các phương pháp đã được phát triển. Dưới đây là một số phương pháp được dùng trong bài báo cáo này:

4.2.1 Phân hoạch Lỗi Xấp xỉ (ACD) của Đa giác

Phương pháp Approximate Convex Decomposition (ACD) giới thiệu một cách tiếp cận khác biệt cơ bản để quản lý sự phức tạp. Thay vì yêu cầu sự lồi hoàn hảo, nó cho phép các thành phần "lồi xấp xỉ" trong một dung sai do người dùng xác định, τ . [9] Điều này thường dẫn đến các phân hoạch có ít thành phần hơn nhiều và phù hợp hơn với các đặc điểm cấu trúc nổi bật của đối tượng.

4.2.1.1 Các Khái niệm Cơ bản trong Phân rã Lồi

Phần này trình bày các khái niệm hình học cơ bản được sử dụng trong quá trình phân rã đa giác không lồi thành các vùng lồi. Các định nghĩa này đóng vai trò nền tảng cho các phương pháp đo độ lồi và các thuật toán phân hoạch ở các phần sau.

- **Bao lồi (Convex Hull – H_P):** Bao lồi của một đa giác P là **tập hợp lồi nhỏ nhất chứa toàn bộ đa giác** đó. Trực quan, có thể hình dung bao lồi như một sợi dây chun được kéo căng bao quanh đa giác. Một đa giác P được gọi là **lồi** nếu và chỉ nếu nó trùng với bao lồi của chính nó, tức là:

$$P = H_P.$$

- **Notches:** Là các đỉnh của đa giác P **không nằm trên bao lồi** H_P . Về mặt hình học, đây là những đỉnh có **góc trong lớn hơn** 180° . Các Notches biểu hiện các đặc trưng lõm (concave features) trên biên của đa giác.
- **Cầu (Bridges):** Là các cạnh của bao lồi ∂H_P nối hai đỉnh không liền kề của biên ngoài ∂P_0 . Về mặt toán học, tập hợp các cầu được định nghĩa:

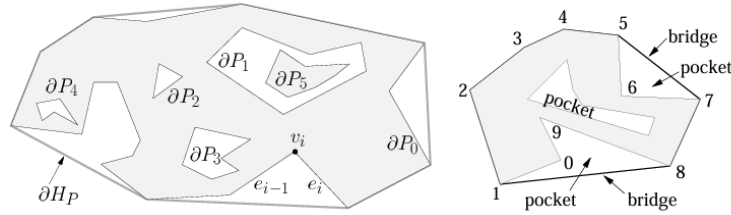
$$\text{BRIDGES}(P) = \partial H_P \setminus \partial P.$$

Mỗi cầu “bắc qua” một vùng lõm của đa giác.

- **Túi (Pockets):** Là các **chuỗi cạnh liên tiếp** của biên đa giác ∂P không thuộc về vỏ lồi ∂H_P . Về mặt toán học:

$$\text{POCKETS}(P) = \partial P \setminus \partial H_P.$$

Mỗi túi tương ứng với một “vịnh lõm” (concave bay) trên đường biên của đa giác.



4.2.1.2 Ý tưởng và Chi tiết Thuật toán

Thuật toán là một chiến lược chia để trị đệ quy[9]:

1. Đo độ lõm: Xác định đặc điểm không lồi (một "notch" hay đỉnh lõm) quan trọng nhất trong đa giác. Đây là điểm có độ lõm lớn nhất. Tùy theo từng trường hợp mà ta có thể sử dụng các thước đo độ lõm khác

nhau, cách để tính những thước đo độ lõm này sẽ được giới thiệu rõ hơn ở phần 4.2.1.3

2. Kiểm tra Dung sai: Nếu độ lõm tối đa đo được nhỏ hơn τ , quá trình kết thúc đối với thành phần đó.
3. Giải quyết Đỉnh lõm: Nếu độ lõm vượt quá τ , một đường cắt được thêm vào để giải quyết đỉnh lõm, chia đa giác thành hai thành phần mới (hoặc hợp nhất một lỗ).

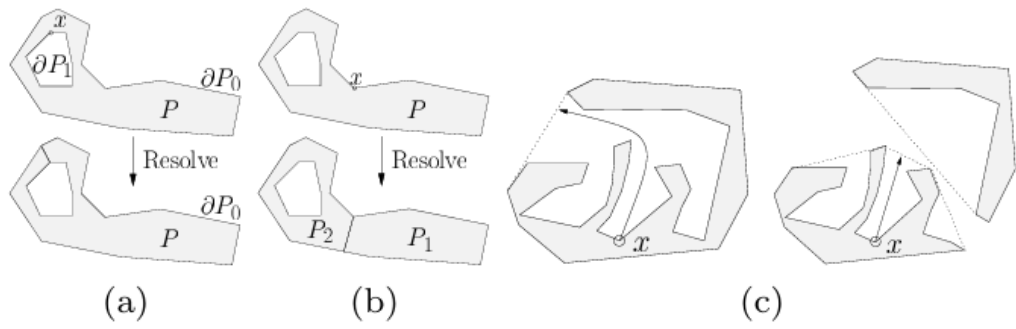


Figure 4.1: (a) Nếu $x \in \partial P_i > 0$, hàm **Resolve** **gộp** biên ∂P_i vào đa giác P_0 . (b) Nếu $x \in \partial P_0$, hàm **Resolve** **tách** đa giác P thành hai đa giác P_1 và P_2 . (c) **Độ lõm** tại điểm x thay đổi sau khi đa giác được phân rã.

4. Độ quy: Quá trình được áp dụng đệ quy cho các thành phần mới.

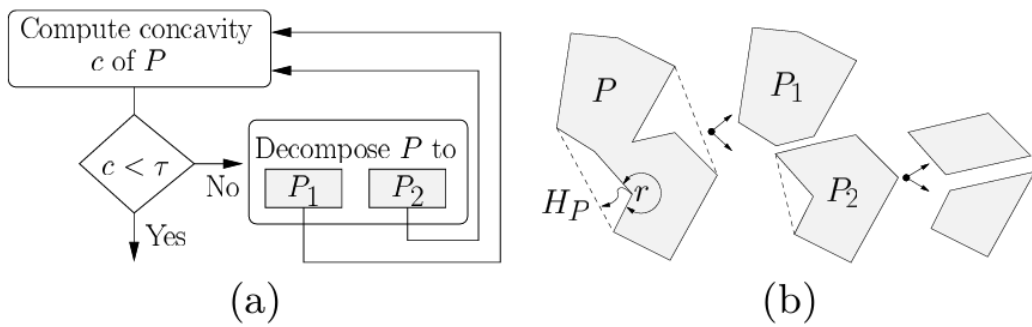


Figure 4.2: Quá trình đệ quy tiếp tục cho đến khi đạt đến giới hạn độ lõm đầu vào của người dùng τ .

4.2.1.3 Các Thước đo Độ lõm

Chất lượng của phân hoạch phụ thuộc rất nhiều vào **cách đo độ lõm** của các đỉnh trong đa giác không lồi. Các phương pháp đo độ lõm phổ biến bao gồm:

- **Độ lõm Đường thẳng (Straight-Line Concavity – SL-Concavity):**

Là khoảng cách Euclid từ một đỉnh trong *túi* (pocket) đến *cầu nối* (bridge) liên kết của nó — thường là một cạnh thuộc bao lồi của đa giác. Phương pháp này rất nhanh, nhưng **không đảm bảo tính đơn điệu**, có thể khiến một số đặc trưng lõm quan trọng bị bỏ sót.



Figure 4.3: Giả sử r là điểm lõm có độ lõm lớn nhất. Sau khi giải quyết (resolve) r , độ lõm của s tăng lên. Nếu $\text{concave}(r) < \tau$, vùng s sẽ không bao giờ được xử lý, ngay cả khi $\text{concave}(s)$ lớn hơn τ .

- **Độ lõm Đường đi Ngắn nhất (Shortest Path Concavity – SP-Concavity):** Là **chiều dài đường đi ngắn nhất** từ một đỉnh trong túi đến cầu nối tương ứng, với điều kiện đường đi đó phải nằm hoàn toàn trong túi. Phương pháp này chậm hơn SL-Concavity nhưng **đảm bảo tính đơn điệu**: độ lõm của các đỉnh giảm dần sau mỗi lần phân rã, giúp đảm bảo rằng tất cả các đặc điểm lõm quan trọng cuối cùng sẽ được xử lý. (Xem chi tiết trong Phần ??.)
- **Độ lõm Lai (Hybrid Concavity – H-Concavity):** Một giải pháp trung gian thực tiễn — sử dụng SL-Concavity nhanh làm mặc định và chỉ chuyển sang SP-Concavity khi **phát hiện khả năng không đơn điệu**. Bằng cách thêm bước kiểm tra *nguy cơ tiềm ẩn* bằng cách so sánh n_β và n_i ; nếu tồn tại $n_i \cdot n_\beta < 0$, biên túi bị đảo hướng — dấu

hiệu của túi phức tạp mà SL-Concavity có thể thất bại.

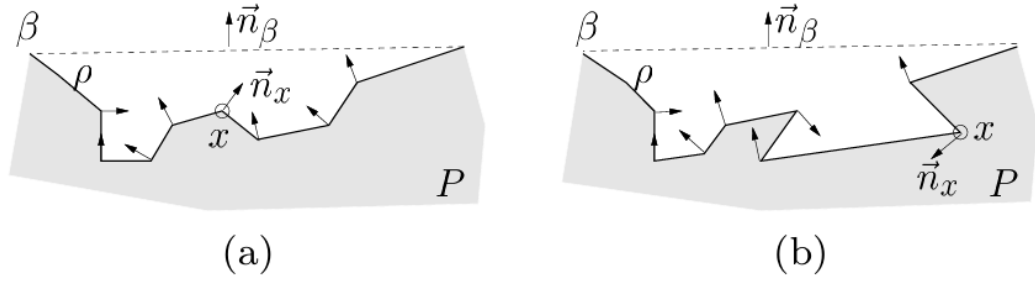


Figure 4.4: Độ lõm SL có thể xử lý được túi trong (a) một cách chính xác vì không có hướng chuẩn nào của các đỉnh trong túi ngược với hướng chuẩn của cầu. Tuy nhiên, túi trong (b) có thể dẫn đến độ lõm giảm không đơn điệu.

4.2.1.4 Đặc tính của Thuật toán

- Loại thuật toán: Đây là một phương pháp xấp xỉ theo định nghĩa. Một phân hoạch với $\tau = 0$ là một phân hoạch lỗi chính xác.
- Độ phức tạp thời gian: $O(nr^2)$, trong đó n là số đỉnh và r là số đỉnh lõm.[9] Điều này nhanh hơn các thuật toán phân hoạch chính xác tối ưu cho đa giác không có lỗ.

4.2.2 Iterative Regional Inflation by Semi-Definite Programming (IRIS)

Thuật toán Iterative Regional Inflation by Semidefinite Programming (IRIS) thay vì cố gắng bao phủ toàn bộ không gian tự do cùng một lúc, nó đặt ra câu hỏi: "Với một điểm 'mầm' (seed) ban đầu, vùng lõi lớn nhất của không gian tự do mà tôi có thể tìm thấy xung quanh nó là gì?". Tức sau khi chạy xong giải thuật output của ta là một vùng lõi lớn nhất có thể của thuật toán từ 1 seed point cho trước.

4.2.2.1 Ý tưởng và Chi tiết Thuật toán

IRIS là một quy trình tối ưu hóa lặp đi lặp lại gồm 4 bước:

1. **Khởi tạo:** Thuật toán sẽ tạo ra một hình elip ban đầu là một hình cầu rất nhỏ có tâm chính là điểm mầm (seed point).
2. **Tìm Siêu phẳng Phân cách (QP):**

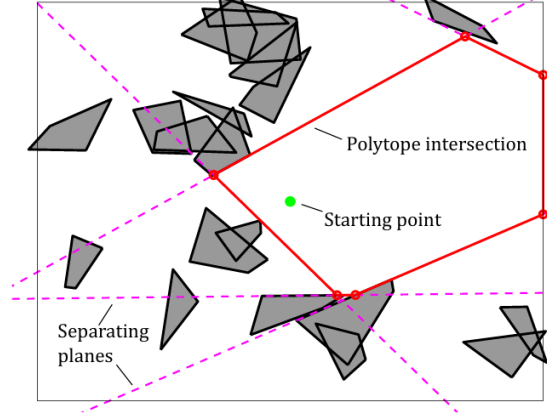


Figure 4.5: Siêu phẳng Phân cách

Đối với ellipsoid hiện tại, thuật toán tìm điểm gần nhất trên mỗi chướng ngại vật. Sau đó, nó xây dựng các siêu phẳng phân cách ellipsoid khỏi các chướng ngại vật này. Mỗi siêu phẳng tiếp tuyến với một phiên bản mở rộng của ellipsoid và đi qua điểm gần nhất trên chướng ngại vật tương ứng. Việc tìm điểm gần nhất trên mỗi chướng ngại vật lỗi được xây dựng như một bài toán Quy hoạch Toàn phương (Quadratic Program - QP). Để giữ cho bài toán tối ưu hóa tiếp theo có quy mô nhỏ, các siêu phẳng thừa (những siêu phẳng không xác định biên của vùng lỗi kết quả) sẽ được loại bỏ.

Bước 2.1. Tìm điểm gần nhất trên obstacle đến hình elip hiện tại

- **Biến đổi không gian (Từ không gian Ellipsoid sang không gian Quả cầu đơn vị)**

Việc tính toán khoảng cách đến một ellipsoid là phức tạp. Tuy nhiên, tính khoảng cách đến một quả cầu đơn vị tại gốc tọa độ thì lại rất đơn giản. Ý tưởng ở đây là biến đổi toàn bộ không gian sao cho ellipsoid trở thành một quả cầu đơn vị.

- Một ellipsoid E được định nghĩa là ảnh của một quả cầu đơn vị $\tilde{E}$ qua phép biến đổi affine:

$$x = C\tilde{x} + d.$$

- Sử dụng phép biến đổi ngược:

$$\tilde{x} = C^{-1}(x - d),$$

để ánh xạ mọi điểm trở lại “không gian quả cầu đơn vị”.

- Trong không gian mới này, ellipsoid trở thành quả cầu đơn vị $\tilde{E}$ tại gốc tọa độ, và mỗi chương ngại vật l_j cũng bị biến đổi (méo đi) thành một đối tượng mới $\tilde{l}_j$.

Bước 2.2: Tìm điểm gần nhất bằng Quy hoạch Toàn phương (QP)

Bây giờ, bài toán tìm điểm trên chương ngại vật l_j gần ellipsoid E nhất tương đương với việc tìm điểm trên chương ngại vật đã biến đổi $\tilde{l}_j$ gần gốc tọa độ nhất.

Bài toán này được xây dựng dưới dạng một bài toán *Quy hoạch Toàn phương* (Quadratic Program - QP):

$$\begin{aligned} & \underset{\tilde{x} \in \mathbb{R}^n, w \in \mathbb{R}^m}{\text{minimize}} && \|\tilde{x}\|^2 \\ & \text{subject to} && \begin{cases} [\tilde{v}_{j,1} \ \tilde{v}_{j,2} \ \cdots \ \tilde{v}_{j,m}] w = \tilde{x}, \\ \sum_{i=1}^m w_i = 1, \\ w_i \geq 0, \quad i = 1, \dots, m. \end{cases} \end{aligned}$$

Về bản chất, ta đang tìm một điểm $\tilde{x}$ là **tổ hợp lồi** của các đỉnh đã biến đổi $\tilde{v}_{j,k}$ sao cho khoảng cách của $\tilde{x}$ đến gốc tọa độ là nhỏ nhất.

Bước 2.3. Xác định mặt phẳng tiếp tuyến của ellipsoid tại điểm tiếp xúc

Sau khi tìm được điểm x^* gần nhất, ta xây dựng mặt phẳng tiếp tuyến với ellipsoid tại điểm đó. Ellipsoid được mô tả bằng biểu thức nghịch đảo:

$$E = \{x \mid (x - d)^\top C^{-1} C^{-\top} (x - d) \leq 1\}.$$

Vector pháp tuyến của ellipsoid tại x^* được xác định bởi gradient:

$$a_j = \nabla_x \left[(x - d)^\top C^{-1} C^{-\top} (x - d) \right]_{x=x^*} = 2C^{-1} C^{-\top} (x^* - d).$$

Vì mặt phẳng tiếp tuyến đi qua x^* , ta có:

$$b_j = a_j^\top x^*.$$

Khi đó, phương trình mặt phẳng tiếp tuyến là:

$$a_j^\top x = b_j,$$

được thêm vào như một **ràng buộc tuyến tính** trong tập khả thi của thuật toán IRIS. Mặt phẳng này đảm bảo ellipsoid mở rộng tối đa nhưng không giao với chướng ngại vật.

Bước 2.4. Lặp lại với tất cả các obstacles trong không gian. Ta sẽ có được một tập các siêu phẳng để tạo thành một vùng lồi đa giác.

3. **Tìm Ellipsoid Nội tiếp Cực đại (SDP):** Giao của các nửa không gian được xác định bởi các siêu phẳng tạo thành một đa diện lồi (polytope). Thuật toán sau đó tìm ellipsoid có thể tích lớn nhất có thể nội tiếp trong đa diện này.

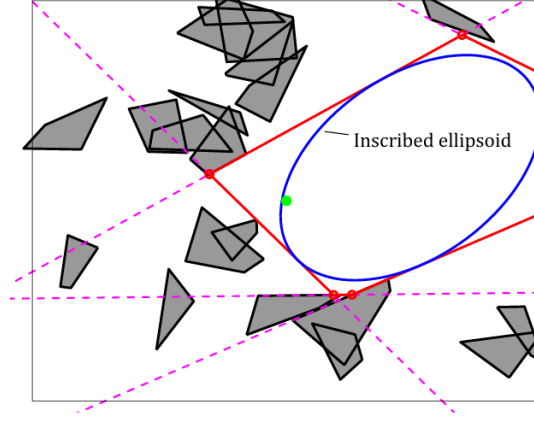


Figure 4.6: Ellipsoid Nội tiếp Cực đại

Bài toán này tìm **ellipsoid có thể tích lớn nhất** nằm gọn trong một đa diện lồi P :

$$P = \{x \in \mathbb{R}^n \mid Ax \leq b\}. \quad (4.1)$$

Ellipsoid được định nghĩa là $\mathcal{E}$, với d là tâm và ma trận C xác định hình dạng:

$$\mathcal{E} = \{C\tilde{x} + d \mid \|\tilde{x}\|_2 \leq 1\}. \quad (4.2)$$

Thể tích của ellipsoid tỉ lệ với $\det(C)$.

Để tìm ellipsoid lớn nhất nội tiếp trong P , ta giải bài toán tối ưu lồi sau:

$$\begin{aligned} & \underset{C, d}{\text{maximize}} && \log \det C \\ & \text{subject to} && \|a_i^\top C\|_2 + a_i^\top d \leq b_i, \quad \forall i = 1, \dots, N, \\ & && C \succ 0. \end{aligned} \quad (4.3)$$

Mục tiêu: Cực đại hoá $\log \det C$ để tối đa hoá thể tích ellipsoid.

Ràng buộc: Đảm bảo mọi điểm của ellipsoid đều nằm trong đa diện

P .

4. **Lặp lại:** Ellipsoid mới, lớn hơn này được sử dụng làm điểm khởi đầu cho vòng lặp tiếp theo. Quá trình này hội tụ khi thể tích của ellipsoid ngừng tăng một cách đáng kể, trả về một đa diện lồi lớn và ellipsoid nội tiếp của nó.

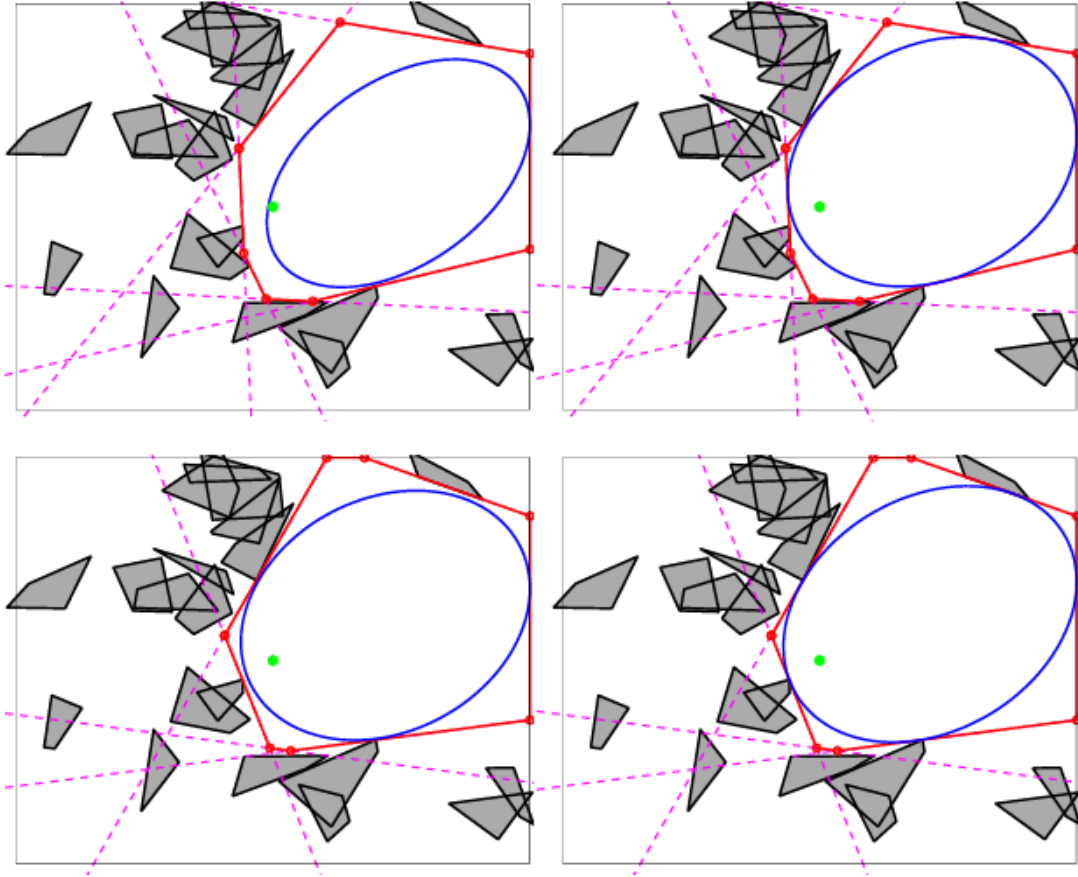


Figure 4.7: Lặp lại

4.2.2.2 Đặc tính của Thuật toán

- Loại thuật toán: IRIS là một phương pháp xấp xỉ để bao phủ toàn bộ không gian tự do. Một lần chạy duy nhất chỉ tạo ra một vùng lõi. Để có được một lớp phủ hoàn chỉnh, cần phải chạy thuật toán nhiều lần với các điểm mầm khác nhau.
- Độ phức tạp thời gian: IRIS thường hội tụ qua 4-8 vòng lặp. Thời gian tính toán của mỗi vòng lặp bằng thời gian tìm các mặt phẳng phân cách cộng với thời gian tính toán ellipses nội tiếp. Trong khi thời gian giải ellipses (SDP) gần như không đổi (constant) nhờ vào việc loại bỏ các siêu phẳng thừa thì thời gian tính các siêu phẳng phân cách lại tăng tuyến tính theo số lượng các obstacles.

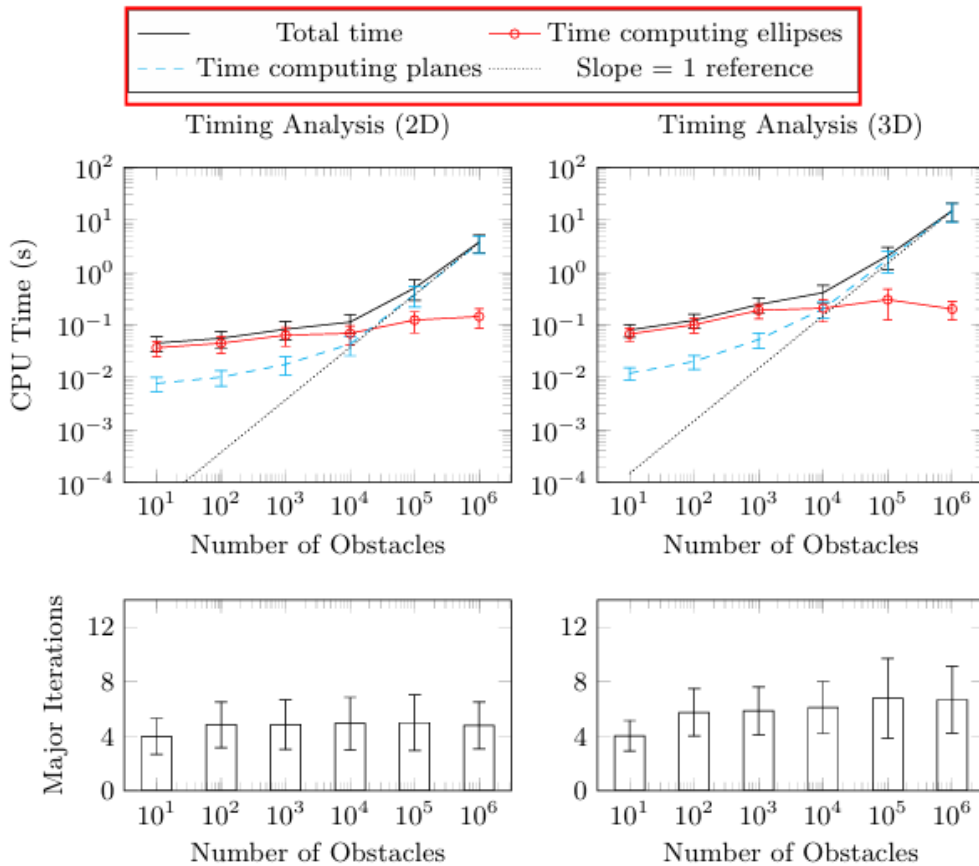


Figure 4.8: Độ phức tạp thời gian

- Xử lý Vật cản không lõi:

- Chướng ngại vật trong không gian work space: Thuật toán IRIS ban đầu giả định các chướng ngại vật là lỗi, đây cũng là một nhược điểm lớn nhất của thuật toán này. Để đảm bảo các obstacles là lỗi, ta thường phải tiền xử lý bằng cách bao lỗi các chướng ngại vật không lỗi[11]. Ngoài ra cách tiếp cận tiêu chuẩn cho các chướng ngại vật không lỗi là phân hoạch chúng thành một tập hợp các mảnh lỗi trước khi chạy IRIS (tức tăng số lượng obstacles của không gian lên). Điều này khả thi vì IRIS có khả năng mở rộng hiệu quả với số lượng lớn chướng ngại vật.
- Chướng ngại vật trong không gian cấu hình (Configuration Space): Đối với các robot có động học phức tạp, nơi các chướng ngại vật trong không gian cấu hình được định nghĩa một cách ẩn ý và không lỗi, các phần mở rộng như IRIS-NP và C-IRIS được sử dụng. Đặc biệt, bài báo "Motion Planning around Obstacles with Convex Optimization" sử dụng một triển khai có tên là IrisInConfigurationSpace trong thư viện Drake cho ví dụ về cánh tay robot.[1] Phần mở rộng này sử dụng lập trình phi tuyến (non-linear programming) để xử lý hình học phức tạp của không gian cấu hình.

4.2.3 Gieo mầm Tự động qua Lớp phủ Clique Khả kiến (VCC)

Thuật toán Visibility Clique Cover (VCC) giải quyết trực tiếp một hạn chế chính của IRIS: nhu cầu về các điểm mầm tốt.⁵ Việc lấy mầm ngẫu nhiên là không hiệu quả. VCC tự động hóa quá trình này bằng cách trước tiên tìm hiểu cấu trúc toàn cục của không gian tự do và sau đó đặt các điểm mầm một cách chiến lược vào các khu vực có khả năng mở rộng lớn.

4.2.3.1 Ý tưởng và Chi tiết Thuật toán

Quy trình VCC bao gồm bốn bước chính [10]:

1. Lấy mẫu & Xây dựng Đồ thị Khả kiến (Visibility Graphs): Thuật toán lấy mẫu n điểm ngẫu nhiên trong không gian cấu hình tự do (C_{free}). Các điểm này tạo thành các đỉnh của một đồ thị khả kiến (visibility graph)¹. Một cạnh tồn tại giữa hai đỉnh nếu đoạn thẳng nối hai điểm tương ứng hoàn toàn nằm trong không gian tự do (tức là không va chạm).

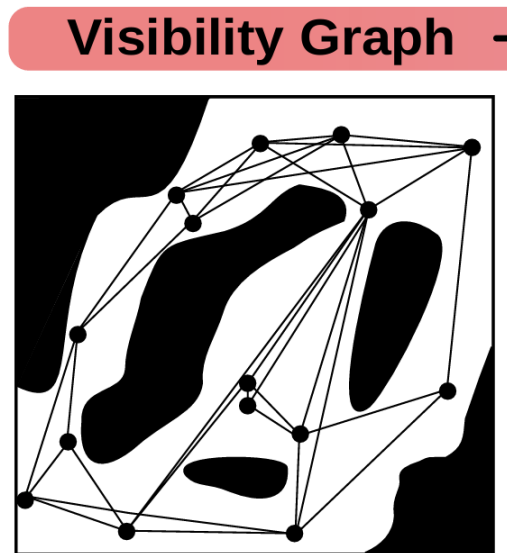


Figure 4.9: Lấy mẫu & Xây dựng Đồ thị Khả kiến (Visibility Graphs)

2. Tìm Lớp phủ Clique: Thuật toán tìm một tập hợp các clique lớn (đồ thị con đầy đủ) trong đồ thị khả kiến. Ý tưởng cốt lõi là một tập hợp các điểm mà tất cả đều "nhìn thấy" nhau có khả năng nằm trong cùng một vùng lồi². Điều này được thực hiện một cách tham lam bằng cách giải lặp đi lặp lại bài toán MAXCLIQUE (tìm clique lớn nhất), được xây dựng dưới dạng một bài toán Integer Linear Programming - ILP.

¹ *Visibility graph* là đồ thị trong đó các đỉnh biểu diễn vị trí (hoặc đỉnh chướng ngại vật), và có cạnh nối giữa hai đỉnh nếu đoạn thẳng nối chúng nằm hoàn toàn trong không gian tự do.

² Nếu hai điểm có thể nhìn thấy nhau, điều đó ngụ ý rằng con đường thẳng nhất giữa chúng là an toàn. Đây là khối xây dựng cơ bản để hiểu và xấp xỉ các vùng lồi trong.

► Clique Cover —

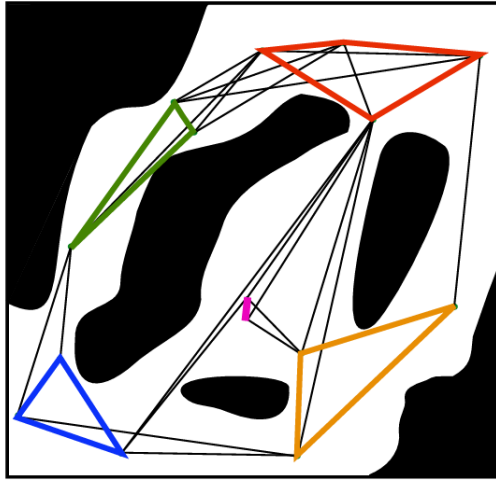


Figure 4.10: Tìm Lớp phủ Clique

3. Khởi tạo Ellipsoid: Đối với mỗi clique được tìm thấy, thuật toán tính toán ellipsoid có thể tích nhỏ nhất bao quanh tất cả các điểm trong clique đó. Ellipsoid này cung cấp một tâm (điểm mầm) và các trục chính (hình dạng ban đầu) cho bước lấp đầy tiếp theo.

► Ellipsoids —

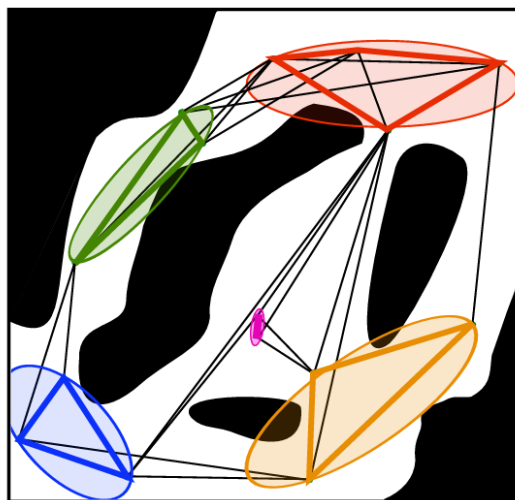


Figure 4.11: Khởi tạo Ellipsoid

4. Lấp đầy thành Đa diện: Tâm và hình dạng của mỗi ellipsoid được sử dụng để khởi tạo một vòng lặp duy nhất của thuật toán lấp đầy tương

tự IRIS. Điều này hiệu quả hơn IRIS tiêu chuẩn vì ellipsoid ban đầu đã được "thông báo" về hình học cục bộ, thường loại bỏ sự cần thiết của nhiều vòng lặp tốn kém.⁵

► Inflate Polytopes -

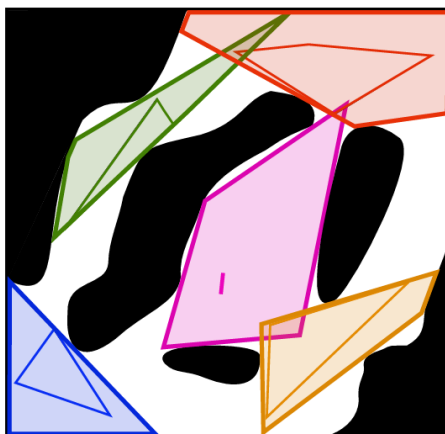


Figure 4.12: Lấp đầy thành Đa diện

5. Lặp lại và Hội tụ: Quá trình này được lặp lại cho đến khi đạt được độ phủ đủ bằng cách lấy mẫu mới từ không gian trống còn lại và lặp lại các bước trước đó.

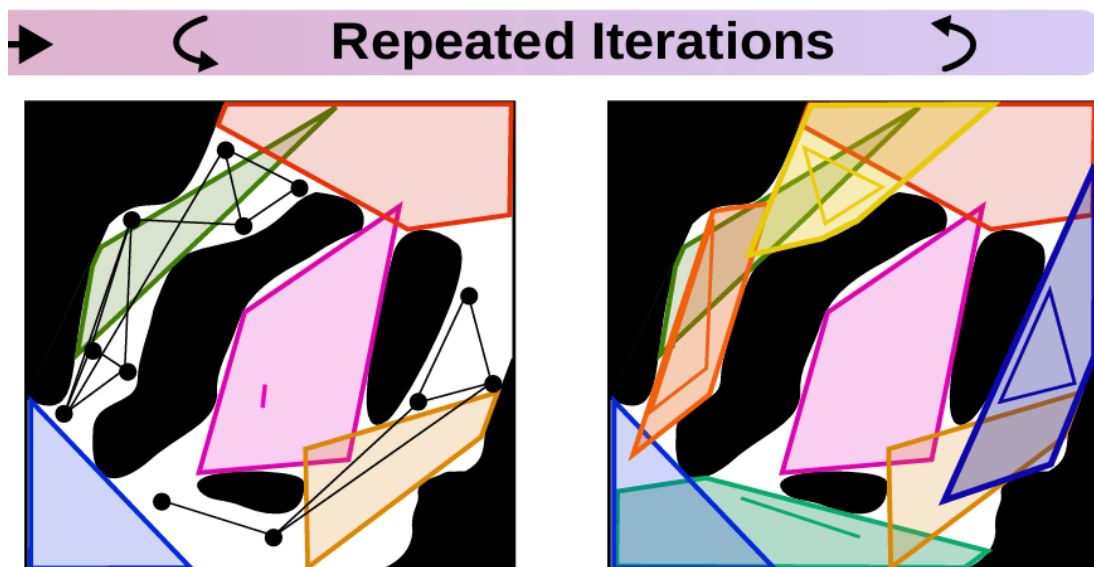


Figure 4.13: Lặp lại

4.2.3.2 Đặc tính và Tính tối ưu

- Loại thuật toán: VCC là một phương pháp xấp xỉ. Chất lượng của lớp phủ được đo bằng tỷ lệ phần trăm thể tích của C_{free} được bao phủ bởi hợp của các đa diện kết quả.⁵
- Mục tiêu Tối ưu: Thuật toán nhằm tìm một lớp phủ lồi xấp xỉ α (α -approximate convex cover) với số lượng vùng tối thiểu. Nó sử dụng bài toán lớp phủ clique tối thiểu (minimum clique cover) như một bài toán tương tự rời rạc cho bài toán lớp phủ lồi tối thiểu.[10]
- Phương pháp: Đây là một cách tiếp cận lai ghép giữa hình học và ILP.
- Xử lý Không gian không lồi: VCC không phụ thuộc vào hình dạng của chướng ngại vật vì nó dựa vào một bộ kiểm tra va chạm chung để kiểm tra khả kiến và sau đó sử dụng một thuật toán lấp đầy cơ bản (như IRIS-NP) có thể xử lý các không gian cấu hình không lồi.[10]

4.3 Graphs of Convex Sets

The Graph of Convex Sets (GCS) framework offers a unified methodology for addressing hybrid discrete-continuous optimization problems. It effectively bridges the gap between combinatorial graph search and continuous convex programming. Unlike classical graph theory, GCS generalizes the Shortest Path Problem (SPP) by associating continuous geometric variables directly with the graph topology.

4.3.1 Shortest Path Formulation in Graph of Convex Sets

A GCS is defined as a directed graph $G = (V, E)$, where each vertex $v \in V$ is associated with a continuous decision variable x_v confined within a non-

empty, compact convex set $X_v \subset \mathbb{R}^n$. In the context of motion planning, these sets typically represent pre-computed collision-free regions within the configuration space. Transitions between vertices are governed by edges $e = (u, v) \in E$, which impose auxiliary convex constraints $(x_u, x_v) \in X_e$ and are weighted by a convex cost function $\ell_e(x_u, x_v)$. This function is required to be proper, closed, and convex, explicitly coupling the continuous variables x_u and x_v to model transition metrics such as Euclidean distance or energy expenditure. It is important to note that, despite the terminology, we do not assume ℓ_e to be a valid metric; properties such as symmetry or the triangle inequality are not required to hold [12].

While the classical network flow formulation effectively solves the SPP on discrete graphs with static edge costs, the GCS framework generalizes this by introducing continuous decision variables coupled with the graph topology. In GCS, the cost of traversing an edge $e = (u, v)$ is not a constant scalar c_{uv} , but a convex function $\ell_e(x_u, x_v)$ dependent on the continuous states $x_u \in X_u$ and $x_v \in X_v$ at the incident vertices. The objective of the SPP in GCS is to identify a path p from a source vertex s to a target vertex t that minimizes the aggregate cost of traversing the edges along the path, while simultaneously selecting appropriate continuous variables at each vertex. Formally, this problem is expressed as:

$$\min_{p, \{x_v\}_{v \in p}} \sum_{(u,v) \in E_p} \ell_{(u,v)}(x_u, x_v) \quad (4.4)$$

$$\text{s.t. } p \in \mathcal{P}, \quad (4.5)$$

$$x_v \in X_v, \quad \forall v \in p, \quad (4.6)$$

$$(x_u, x_v) \in X_e, \quad \forall e = (u, v) \in E_p \quad (4.7)$$

where $\mathcal{P}$ denotes the set of all feasible paths from s to t , and E_p represents the edges included in the path p .

By applying the network flow formulation to GCS, the SPP can be represented as a Mixed-Integer Nonlinear Program (MINLP) as follows:

$$\text{minimize} \quad \sum_{e=(u,v) \in \mathcal{E}} y_e \ell_e(x_u, x_v) \quad (4.8a)$$

$$\text{subject to} \quad \sum_{e \in \mathcal{E}^+(u)} y_e - \sum_{e \in \mathcal{E}^-(u)} y_e = \delta_u, \quad \forall u \in \mathcal{V} \quad (4.8b)$$

$$(x_u, x_v) \in X_e, \quad \forall e = (u, v) \in \mathcal{E} \quad (4.8c)$$

$$x_v \in X_v, \quad \forall v \in \mathcal{V} \quad (4.8d)$$

$$y_e \in \{0, 1\}, \quad \forall e \in \mathcal{E} \quad (4.8e)$$

In this formulation, the binary variable y_e indicates whether edge e is included in the selected path ($y_e = 1$) or not ($y_e = 0$). The flow conservation constraint (4.8b) ensures that the selected edges form a valid path from the source vertex s to the target vertex t , where δ_u is defined as 1 for the source, -1 for the target, and 0 for all other vertices. Constraints (4.8c) and (4.8d) enforce that the continuous variables at each vertex and along each edge adhere to their respective convex sets.

4.3.2 Complexity Analysis

The computational complexity of the SPP in GCS differs fundamentally from that of the classical Shortest Path Problem on discrete graphs. In classical graph theory, the SPP is generally NP-hard (e.g., the Longest Path problem or variants involving negative cycles). However, it becomes efficiently solvable in polynomial time under two critical conditions:

1. The graph is **acyclic** (allowing for topological sort-based solutions).
2. All edge and vertex costs are **non-negative** (solvable via Dijkstra's algorithm).

Proposition 1. *In stark contrast, the Shortest Path Problem in a Graph*

of *Convex Sets* remains **NP-hard** even when both of the aforementioned conditions—acyclicity and non-negative costs—are satisfied.

This hardness is established via a reduction from the **3SAT** problem (Boolean satisfiability with clauses of three literals), which is known to be NP-complete. As demonstrated in Theorem 9.1 of [13], one can construct a layered, acyclic GCS instance with non-negative costs where identifying a valid path is equivalent to finding a satisfying assignment for a 3SAT formula. This implies that merely determining the *feasibility* of an SPP in GCS is NP-hard. Alternative proofs of NP-hardness have also been provided for cases involving low-dimensional convex sets (1D and 2D), though these instances typically rely on graphs containing cycles [13].

4.3.3 Detailed Formulation of Vertices and Edges

In the specific context of trajectory optimization, the components of the GCS are rigorously defined to enforce both geometric safety and kinematic feasibility.

Each vertex $v \in V$ corresponds to a specific convex, collision-free region Q_v within the Configuration Space (C-space). The continuous decision variable $x_v \in \mathbb{R}^{n_v}$ associated with vertex v represents a local trajectory segment or a state configuration. It is crucial to note that the dimension n_v is local to the vertex, allowing for variable dimensionality across the graph.

The variable x_v is constrained to lie within a compact convex set $\mathcal{X}_v \subset \mathbb{R}^{n_v}$. For trajectory generation using piecewise polynomial parameterizations (e.g., Bézier curves), $\mathcal{X}_v$ is explicitly constructed to ensure the curve lies within the safe region Q_v and satisfies dynamic limits. The defining constraints for $\mathcal{X}_v$ are typically formulated as:

$$r_{i,k} \in Q_i, \quad k = 0, \dots, d, \quad (4.9)$$

$$\dot{h}_{i,k} \geq \dot{h}_{\min} = 10^{-6}, \quad k = 0, \dots, d-1, \quad (4.10)$$

$$\dot{r}_{i,k} \in \dot{h}_{i,k} D, \quad k = 0, \dots, d-1, \quad (4.11)$$

$$h_{i,0} \geq 0, \quad h_{i,d} \leq T_{\max}. \quad (4.12)$$

Here, (4.9) enforces geometric containment of the spatial control points $r_{i,k}$ within the safe region. Equation (4.10) ensures strict monotonicity of the time-scaling function to preserve causality. Equation (4.11) couples spatial and temporal derivatives to satisfy velocity bounds defined by the set D , and (4.12) imposes limits on the trajectory duration.

An edge $e = (u, v) \in E$ defines the transition between two vertices. This transition is governed by a *Coupling Constraint Set* $\mathcal{X}_e \subseteq \mathbb{R}^{n_u+n_v}$. This set imposes constraints on the concatenated vector (x_u, x_v) , residing in the Cartesian product of the individual variable spaces. Mathematically, $\mathcal{X}_e$ defines which pairs of configurations (x_u, x_v) are compatible, ensuring continuity (e.g., C^0, C^1 , or higher-order continuity) between trajectory segments.

Associated with each edge is a *Coupling Cost Function* $f_e : \mathbb{R}^{n_u+n_v} \rightarrow \mathbb{R}$. This is a scalar-valued convex function that assigns a cost based on the simultaneous selection of x_u and x_v , modeling transition metrics such as path length or energy consumption.

4.3.4 Cost Function Specifications

The selection of the edge cost function $\ell_e(x_u, x_v)$ is pivotal, as it dictates the physical interpretation of the optimal trajectory. Our framework distinguishes between two primary formulations: geometric path planning (LinearGCS) and kinodynamic trajectory optimization (BezierGCS), each em-

ploying specific convex structures to ensure computational efficiency.

4.3.4.1 Geometric Path Length

For problems strictly defined by geometric constraints, such as identifying the shortest path through convex regions, the cost function is characterized by the weighted Euclidean distance. The squared Euclidean distance,

$$\ell_e(x_u, x_v) = \|x_v - x_u\|_2^2,$$

is a quadratic function that fits naturally into Quadratic Programming (QP) solvers. Alternatively, the standard L_2 -norm

$$\ell_e(x_u, x_v) = \|x_v - x_u\|_2$$

can be modeled using Second-Order Cone Programming (SOCP), preserving convexity. These formulations ensure that the cost scales linearly with the Euclidean distance, yielding a geometrically accurate metric for path length.

4.3.4.2 Dynamic Trajectory Costs

In the proposed motion planning framework utilizing Bézier curves, the continuous decision variable x_v is formulated to encapsulate both the spatial control points and the temporal duration of the trajectory segment. To effectively manage the trade-off between execution speed and control effort, the cost function is constructed as a composite objective comprising three distinct terms.

First, to prioritize time optimality, a linear cost is applied to the time-scaling variables h . This formulation ensures that the minimization of the total trajectory duration preserves the convexity of the objective function.

The time cost is expressed as:

$$J_t = w_t T_e, \quad (4.13)$$

where w_t denotes the weighting factor assigned to the segment duration T_e . Simultaneously, to penalize control effort—specifically minimizing the integral of squared velocity or acceleration—the framework leverages the mathematical properties of perspective functions. For a trajectory segment with duration T_e , this energy cost is defined as:

$$J_e = \int_0^{T_e} |\dot{r}(t)|^2 dt. \quad (4.14)$$

This term behaves as a perspective function of the form quadratic-over-linear ($x^T x/y$). A critical advantage of this formulation is that it is jointly convex with respect to both the spatial control points and the duration variable. Consequently, this allows the solver to optimize the geometric path and the time allocation simultaneously within a unified Mixed-Integer Convex Programming (MICP) framework.

Finally, to ensure higher-order smoothness, such as jerk minimization, the cost function incorporates derivative regularization. Unlike the energy cost, this is achieved by applying standard quadratic costs directly to the coefficients of the trajectory’s higher derivatives. This approach avoids the strict requirements of perspective formulations while successfully maintaining the overall convex structure of the optimization problem.

4.3.5 Mixed-Integer Convex Programming Formulation via Perspective Relaxation

Directly solving the problem as a Mixed-Integer Nonlinear Programming (MINLP) formulation poses significant computational challenges due to the presence of non-convex constraints, particularly the bilinear relation-

ship between the binary decision variables y_e and the continuous state variables x_u . To overcome this and transform the problem into a tractable Mixed-Integer Convex Programming (MICP) form, we apply the perspective relaxation technique. The primary motivation of this method is to isolate the non-convexity by “lifting” the continuous variables into a higher-dimensional space. Instead of utilizing global variables x_u , we introduce local proxy variables $z_{e,u}, z_{e,v} \in \mathbb{R}^n$ for each edge $e = (u, v)$, representing the state contribution of the corresponding vertices when that edge is active. For each edge $e = (u, v)$, the local proxy variables $z_{e,u}$ and $z_{e,v}$ specifically model the contribution of the state at vertices u and v to the traversal of edge e . If the edge is not active (i.e., $y_e = 0$), these contributions are forced to zero. This relationship is expressed as:

$$z_{e,u} \in y_e X_u \quad \text{and} \quad z_{e,v} \in y_e X_v$$

which implies:

$$\begin{cases} z_{e,u} \in X_u & \text{if } y_e = 1 \\ z_{e,u} = 0 & \text{if } y_e = 0 \end{cases}$$

This logic is formalized using the perspective function $\tilde{\ell}_e((z_{e,u}, z_{e,v}), y_e)$, which is the perspective of the original cost function ℓ_e . A fundamental property in convex analysis states that if ℓ_e is convex, its perspective $\tilde{\ell}_e$ is jointly convex in all arguments. This property is crucial as it enables the use of efficient convex solvers. The perspective function is defined as:

$$\tilde{\ell}_e(z_{e,u}, z_{e,v}, y_e) = \begin{cases} y_e \ell_e\left(\frac{z_{e,u}}{y_e}, \frac{z_{e,v}}{y_e}\right) & \text{if } y_e > 0 \\ 0 & \text{if } y_e = 0 \text{ and } z_{e,u} = z_{e,v} = 0 \\ +\infty & \text{otherwise} \end{cases}$$

Crucially, rather than enforcing explicit equality constraints such as $z_{e,u} =$

$y_e x_u$, which are non-convex, we extend the standard flow conservation law (4.8b) to the continuous variables. This mandates that for any node u (excluding the source and target), the sum of continuous states entering the node via incoming edges must equal the sum of continuous states leaving the node via outgoing edges.

The resulting MICP formulation for the Shortest Path Problem in GCS is defined as follows:

$$\text{minimize} \quad \sum_{e=(u,v) \in \mathcal{E}} \tilde{\ell}_e((z_{e,u}, z_{e,v}), y_e) \quad (4.15a)$$

$$\text{subject to} \quad \sum_{e \in \mathcal{E}^+(u)} y_e - \sum_{e \in \mathcal{E}^-(u)} y_e = \delta_u, \quad \forall u \in \mathcal{V} \quad (4.15b)$$

$$\sum_{e \in \mathcal{E}^+(u)} z_{e,u} - \sum_{e \in \mathcal{E}^-(u)} z_{e,v} = \begin{cases} x_s & \text{if } u = s \\ -x_t & \text{if } u = t \\ 0 & \text{otherwise} \end{cases}, \quad \forall u \in \mathcal{V} \quad (4.15c)$$

$$(z_{e,u}, z_{e,v}) \in y_e \mathcal{X}_e, \quad \forall e \in \mathcal{E} \quad (4.15d)$$

$$z_{e,u} \in y_e X_u, \quad z_{e,v} \in y_e X_v, \quad \forall e = (u, v) \in \mathcal{E} \quad (4.15e)$$

$$y_e \in \{0, 1\}, \quad \forall e \in \mathcal{E} \quad (4.15f)$$

In this model, Equation (4.15b) ensures topological connectivity, where δ_u is defined as 1 for the source, -1 for the target, and 0 otherwise. Equation (4.15c) ensures that the continuous trajectory is seamless across the graph structure. Equations (4.15d) and (4.15e) enforce geometric containment within scaled convex sets; specifically, if $y_e = 0$, the associated continuous variables are constrained to the origin, incurring zero cost. This formulation provides a computationally efficient and tight relaxation, allowing the solver to identify near-global optimal trajectories that satisfy

both the discrete graph topology and the complex continuous constraints inherent in motion planning.

4.3.6 Convexification of the GCS Problem via the RLT Technique

To address the inherent non-convexity of the initial Mixed-Integer Non-Convex Program (MINCP)—specifically the bilinear constraints coupling binary decision variables with continuous state variables—we employ a convexification strategy based on the Reformulation-Linearization Technique (RLT) of the first level. The fundamental objective of this approach is to replace intractable constraints with tight convex envelopes. This ensures that the resulting Mixed-Integer Convex Program (MICP) preserves the integer feasible set of the original problem while providing strong lower bounds when the integrality constraints are relaxed.

4.3.6.1 Theoretical Foundation: The Duality between Perspective Cones and Valid Inequalities

Before detailing the optimization model, we must establish the mathematical connection between perspective cones and linear inequalities. Consider the dual cone of the perspective cone $\tilde{X}$, denoted as $(\tilde{X})^*$. This is defined as the set of vectors (a, b) in the extended space such that their inner product with every element in the cone $\tilde{X}$ is non-negative:

$$(\tilde{X})^* := \{(a, b) \in \mathbb{R}^{n+1} \mid (a, b)^\top k \geq 0, \quad \forall k \in \tilde{X}\}. \quad (4.16)$$

A critical result utilized in this formulation is the equivalence between the dual cone $(\tilde{X})^*$ and the set of valid linear inequalities for the set X . Geometrically, this represents the intersection of all closed half-spaces containing X . We formalize this in the following proposition:

Proposition. The dual cone $(\tilde{X})^*$ is precisely the set of coefficients (a, b) that define valid linear inequalities over the set X ; that is, $(\tilde{X})^* \equiv X^\circ$.

Proof. To demonstrate this, consider an arbitrary element $k \in \tilde{X}$. By the definition of a perspective cone, k can be represented as $k = (\lambda x, \lambda)$ for some $x \in X$ and a scalar $\lambda \geq 0$. The condition for a vector pair (a, b) to belong to the dual cone $(\tilde{X})^*$ can be expanded as follows:

$$\begin{aligned} (a, b) \in (\tilde{X})^* &\iff (a, b)^\top (\lambda x, \lambda) \geq 0, \quad \forall x \in X, \forall \lambda \geq 0 \\ &\iff \lambda(a^\top x + b) \geq 0, \quad \forall x \in X, \forall \lambda \geq 0. \end{aligned} \quad (4.17)$$

Observe that the final inequality must hold for all non-negative values of λ . In the case where $\lambda > 0$, we may divide both sides by λ without altering the inequality sign, yielding the condition $a^\top x + b \geq 0$. The case $\lambda = 0$ is trivially satisfied. Consequently, the necessary and sufficient condition for $(a, b) \in (\tilde{X})^*$ is that the linear expression $a^\top x + b$ remains non-negative over the entire set X . This confirms that $(\tilde{X})^*$ constitutes the set of coefficients for all valid inequalities on X , completing the proof. $\square$

4.3.6.2 Construction of Convex Constraints

Leveraging the proposition above, we can “lift” any valid linear constraint from the flow variable space into the mixed state-flow space. Consider an arbitrary vertex v and a valid linear inequality concerning the flow variables y_e incident to v , given by:

$$\sum_{e \in E_v} c_e y_e + d \geq 0, \quad (4.18)$$

where E_v denotes the set of edges adjacent to v , and d is a problem-dependent constant. By applying the perspective principle, we transform the inequality (4.18) into a constraint within the convex cone $\tilde{X}_v$. To facilitate this, we introduce proxy variables z_e and z'_e . The variable z_e represents

the state contribution at the source vertex of edge e when active, while z'_e represents the state contribution at the target vertex.

This transformation yields the following convex cone constraint:

$$\left(\sum_{e \in E_{\text{in}}(v)} c_e z'_e + \sum_{e \in E_{\text{out}}(v)} c_e z_e + dx_v, \sum_{e \in E_v} c_e y_e + d \right) \in \tilde{X}_v. \quad (4.19)$$

Equation (4.19) constrains the relationship between the intermediate state variables (z, z') and the flow variables y , ensuring they reside within the perspective cone of the feasible set X_v .

A specific yet crucial application of this Lemma involves the non-negativity constraint of the flow, i.e., $y_e \geq 0$. For an edge $e = (u, v)$:

- At the source vertex u , applying the lifting with $c_e = 1$ yields the constraint $(z_e, y_e) \in \tilde{X}_u$.
- At the target vertex v , applying the lifting with $c_e = 1$ yields the constraint $(z'_e, y_e) \in \tilde{X}_v$.

These constraints effectively function as an “on/off” mechanism: if $y_e = 1$, the proxy variable z_e is constrained to the set X_u ; conversely, if $y_e = 0$, z_e is forced to the origin.

4.3.6.3 Complete MICP Formulation

Synthesizing the preceding analysis, we propose the Mixed-Integer Convex Programming (MICP) formulation for the Shortest Path Problem on GCS, as described in Theorem 5.7. This model completely eliminates bilinear components, replacing them with convex cone constraints:

$$\underset{y, z, z'}{\text{minimize}} \quad \sum_{e \in E} \tilde{\ell}_e((z_e, z'_e), y_e) \quad (5.5a)$$

$$\text{subject to} \quad \sum_{e \in E_{\text{out}}(s)} y_e = 1, \quad \sum_{e \in E_{\text{in}}(t)} y_e = 1, \quad (5.5b)$$

$$\sum_{e \in E_{\text{out}}(v)} y_e \leq 1, \quad \forall v \in V \setminus \{s, t\}, \quad (5.5c)$$

$$\sum_{e \in E_{\text{in}}(v)} (z'_e, y_e) = \sum_{e \in E_{\text{out}}(v)} (z_e, y_e), \quad \forall v \in V \setminus \{s, t\}, \quad (5.5d)$$

$$(z_e, y_e) \in \tilde{X}_u, \quad (z'_e, y_e) \in \tilde{X}_v, \quad \forall e = (u, v) \in E, \quad (5.5e)$$

$$y_e \in \{0, 1\}, \quad \forall e \in E. \quad (5.5f)$$

The formulation (4.20) is composed of four primary components that ensure graph connectivity and geometric consistency.

First, the objective function (5.5a) represents the total cost across all edges, where $\tilde{\ell}_e$ is the perspective function of the original cost metric. Since perspective operations preserve convexity, the optimization problem maintains a tractable convex structure.

Second, constraints (5.5b) and (5.5c) enforce classical flow conservation, guaranteeing the existence of a path from the source s to the target t while restricting each intermediate vertex to be visited at most once.

Third, constraint (5.5d) represents the spatial flow conservation, a distinct feature compared to standard shortest path problems. This constraint is derived by multiplying the flow conservation equation at vertex v by the state variable x_v . Physically, it ensures trajectory continuity: the aggregated state “entering” vertex v (via z'_e) must equal the aggregated state “leaving” vertex v (via z_e).

Finally, constraints (5.5e) result from the convexification process discussed previously. They serve as the convex envelopes for the proxy variables z_e and z'_e , ensuring that when an edge $e = (u, v)$ is selected ($y_e = 1$), these vari-

ables correspond to valid points within the sets X_u and X_v , respectively.

4.4 Proposed solution

4.5 Experiment setup

4.5.1 Implementation details

4.6 Results and analysis

Chapter 5

Conclusion

5.1 Summary

5.2 Future Work

Future research will focus on...

- ...
- ...

References

- [1] R. Deits and R. Tedrake, “Motion planning around obstacles with convex optimization,” *arXiv preprint arXiv:2205.04422*, 2022.
- [2] Boston Dynamics, *Leaps, bounds, and backflips*, <https://bostondynamics.com/blog/leaps-bounds-and-backflips/>, Accessed: November 29, 2025, 2021.
- [3] Amazon Robotics, *Amazon.com announces first quarter results*, <https://ir.aboutamazon.com/news-release/news-release-details/2023/Amazon.com-Announces-First-Quarter-Results/default.aspx>, Accessed: November 29, 2025, 2023.
- [4] M. Diehl, H. G. Bock, H. Diedam, and P.-B. Wieber, “Fast direct multiple shooting algorithms for optimal robot control,” in *Fast Motions in Biomechanics and Robotics*, Springer, 2006, pp. 65–93.
- [5] S. M. LaValle, “Rapidly-exploring random trees: A new tool for path planning,” Iowa State University, Tech. Rep. TR 98-11, 1998.
- [6] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.
- [7] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004, ISBN: 0521833787.

- [8] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993, ISBN: 0-13-617549-X.
- [9] J. M. Lien and N. M. Amato, “Approximate convex decomposition of polygons,” *Computational Geometry: Theory and Applications*, 2006.
- [10] R. Deits and R. Tedrake, “Approximating robot configuration spaces with few convex sets using clique covers of visibility graphs,” *arXiv preprint arXiv:2310.02875*, 2023.
- [11] R. Deits and R. Tedrake, “Computing large convex regions of obstacle-free space through semidefinite programming,” *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2014.
- [12] T. Marcucci, J. Umenberger, P. A. Parrilo, and R. Tedrake, “Shortest paths in graphs of convex sets,” *arXiv preprint arXiv:2101.11565*, 2023.
- [13] T. Marcucci, R. Tedrake, and P. A. Parrilo, “Graphs of convex sets with applications to optimal control and motion planning,” *MIT Department of Electrical Engineering and Computer Science, PhD Thesis*, 2024.

Appendix A

...

A.1 ...

A.2 ...